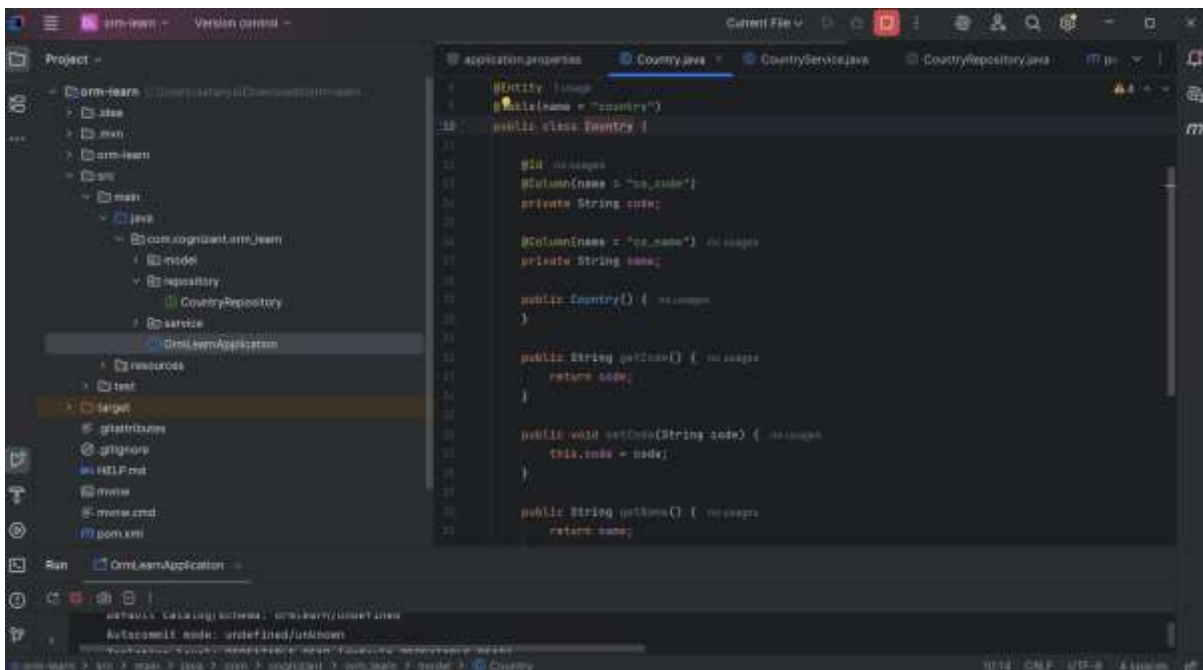
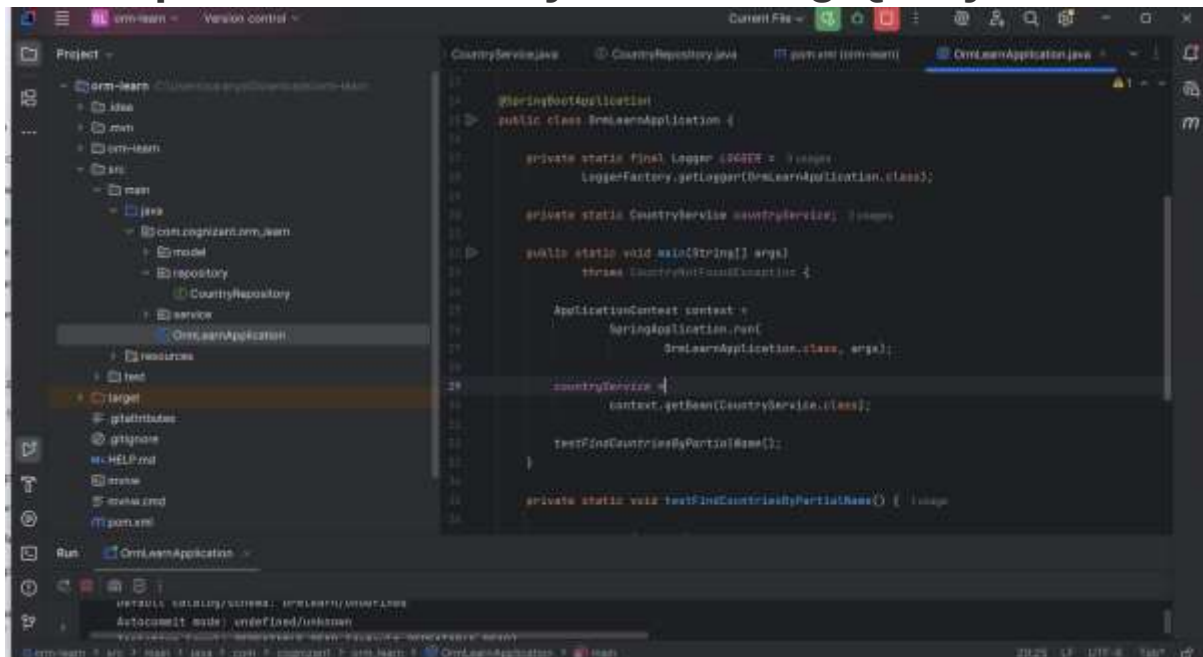
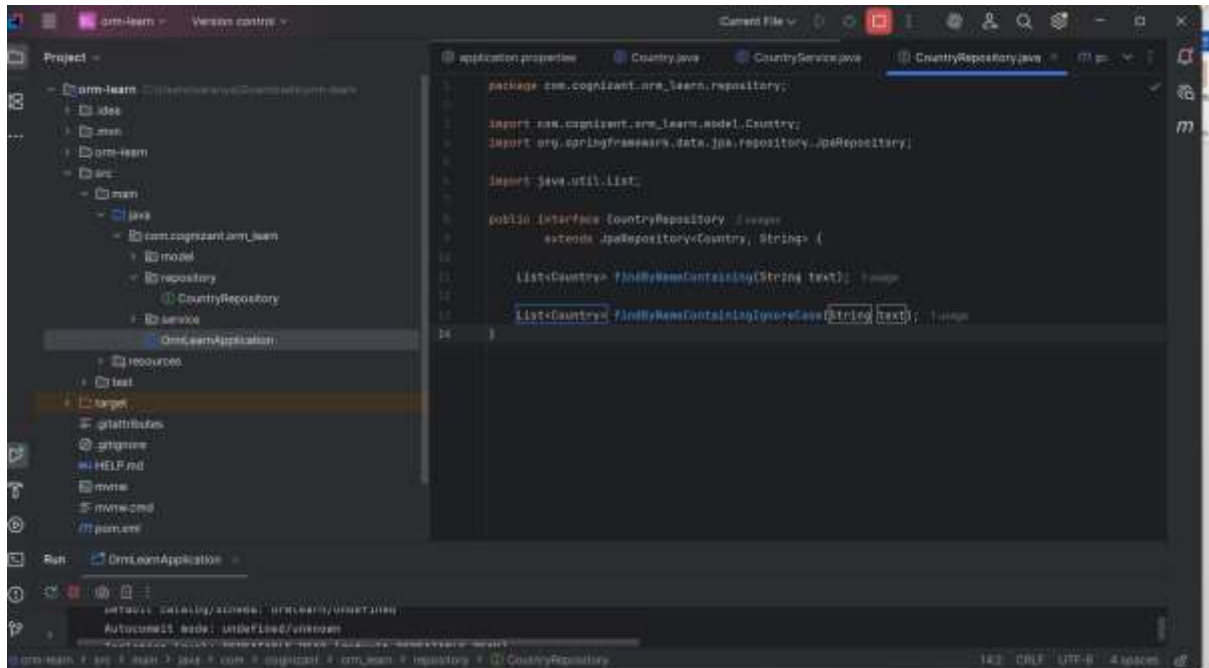
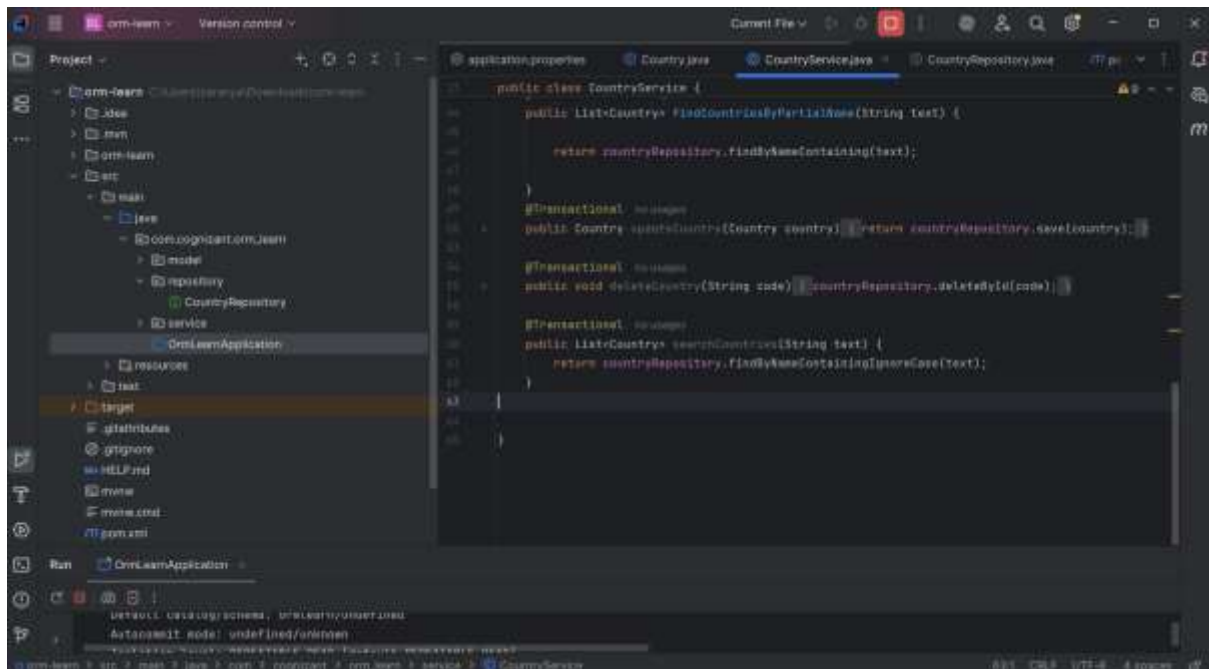


Hands on 1

Write queries on country table using Query Methods





OUTPUT:

```

Database dialect: MySQLDialect
Database version: 8.0.44
Default catalog/schema: orslearn/undefined
Automatic mode: undefined/unknown
Isolation (level): REPEATABLE_READ (default REPEATABLE_READ)
JDBC fetch size: none
Pool: DataSourceConnectionProvider
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown

24-06-26 20:08:31.479 restartedMain INFO org.hibernate.orm.core           initDataService 55 HH0806489: No JTA platform available (set 'hibernat
24-06-26 20:08:31.621 restartedMain INFO org.hibernate.orm.core           buildNativeEntityManagerFactory 464 Initialized JPA EntityManagerFactory for pers
24-06-26 20:08:31.805 restartedMain INFO org.hibernate.orm.core           <init> 48 Hibernate is in classpath; if applicable, HQL parse
24-06-26 20:08:32.505 restartedMain WARN org.hibernate.orm.core           ation$OpenConfiguration openEntityManagerInViewInterceptor 261 spring.jpa.open-in-view is enabled by defe
24-06-26 20:08:32.555 restartedMain INFO org.springframework.boot         start 240 Tomcat started on port 8080 (http) with context pat
24-06-26 20:08:33.466 restartedMain INFO org.springframework.boot         logStarted 60 Started OrslearnApplication in 11.963 seconds (pro
24-06-26 20:08:33.477 restartedMain INFO org.springframework.boot         logStarted 60 Started OrslearnApplication in 11.963 seconds (pro
24-06-26 20:08:34.433 restartedMain INFO org.springframework.boot         logStatement 12A select ci.ci_code,ci.ci_name from country ci
24-06-26 20:08:34.518 restartedMain DEBUG org.springframework.boot         lambda$testFindCountriesByPartialName$ 43 Country:Country {code=GV, name=Guove
24-06-26 20:08:34.519 restartedMain DEBUG org.springframework.boot         lambda$testFindCountriesByPartialName$ 43 Country:Country {code=GP, name=Guade
24-06-26 20:08:34.519 restartedMain DEBUG org.springframework.boot         lambda$testFindCountriesByPartialName$ 43 Country:Country {code=LU, name=Luembe
24-06-26 20:08:34.519 restartedMain DEBUG org.springframework.boot         lambda$testFindCountriesByPartialName$ 43 Country:Country {code=SS, name=South
24-06-26 20:08:34.519 restartedMain DEBUG org.springframework.boot         lambda$testFindCountriesByPartialName$ 43 Country:Country {code=TF, name=French
24-06-26 20:08:34.520 restartedMain DEBUG org.springframework.boot         lambda$testFindCountriesByPartialName$ 43 Country:Country {code=UM, name=United
24-06-26 20:08:34.520 restartedMain INFO org.springframework.boot         testFindCountriesByPartialName 45 End
  
```

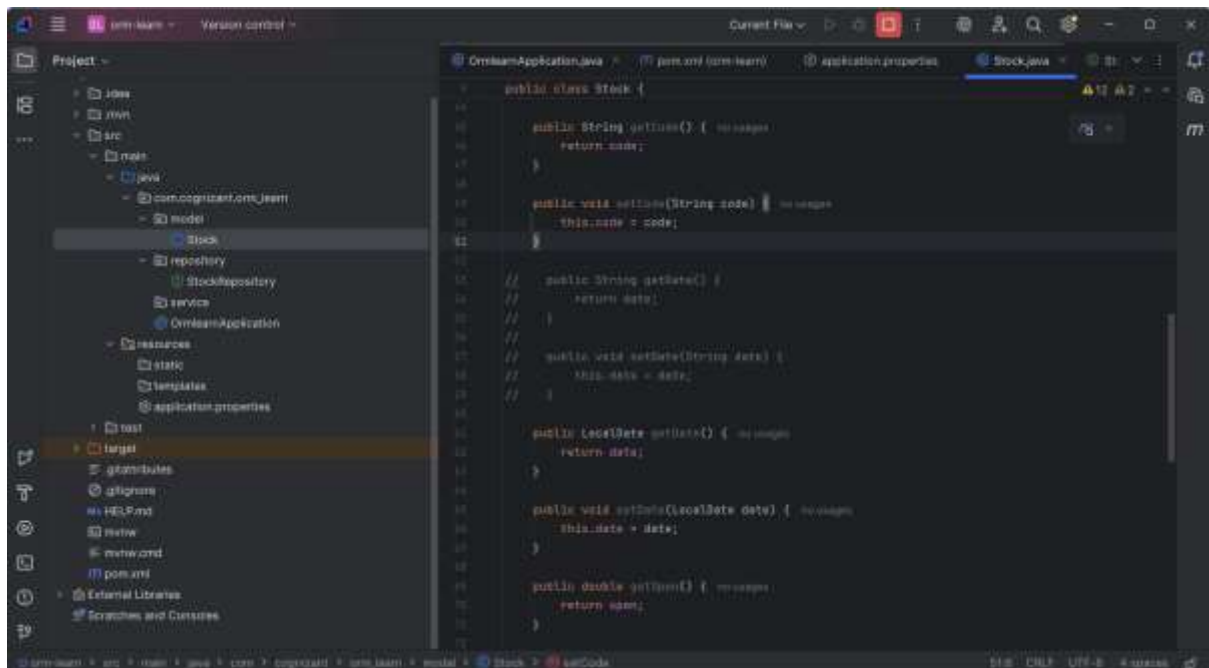
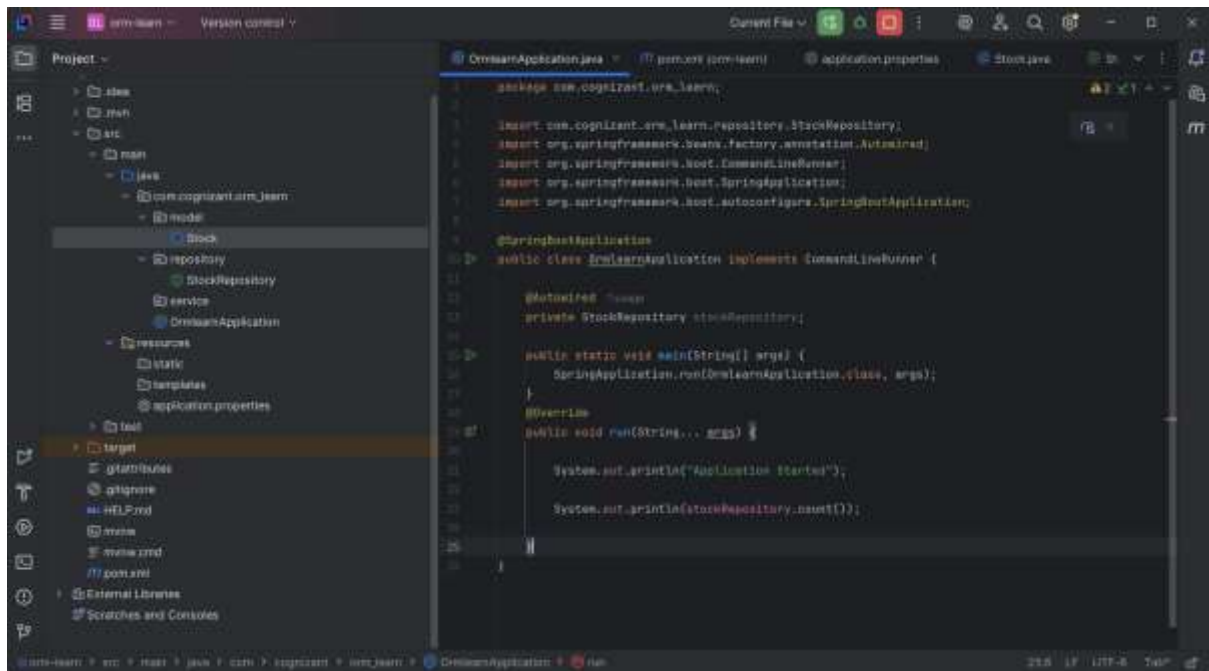
MYSQL(DATABASE):

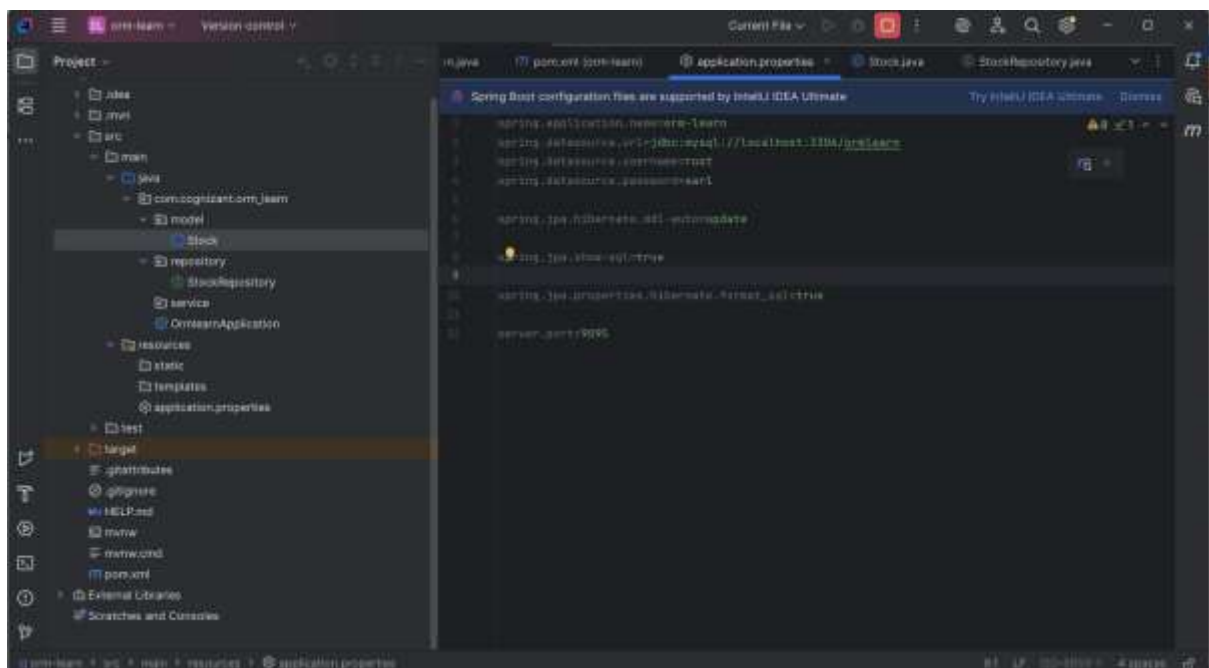
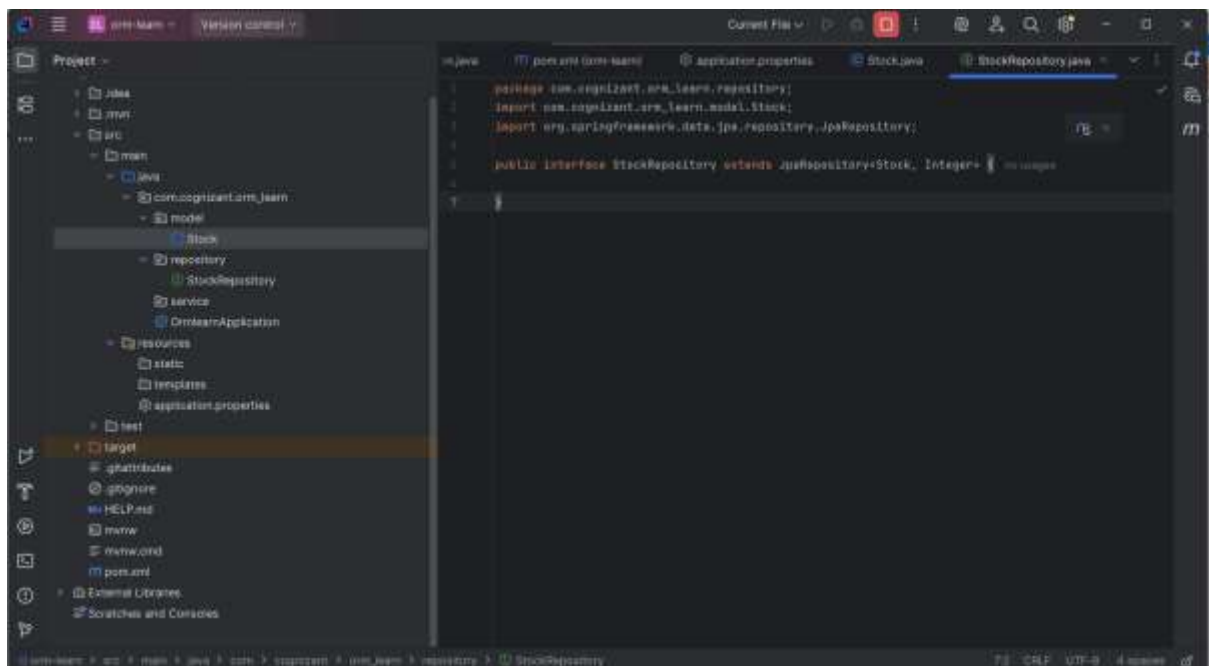
```

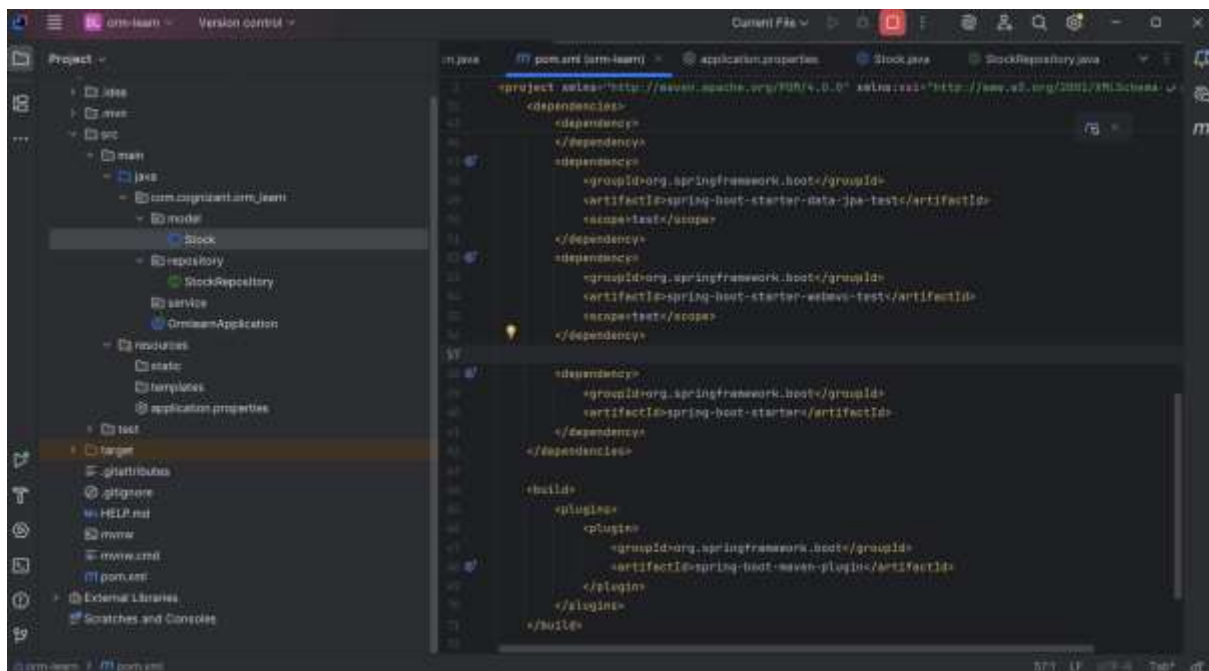
240 * insert into country (ci_code, ci_name) values ("VI", "Virt Nam");
250 * insert into country (ci_code, ci_name) values ("VI", "Virgils Islands, British");
251 * insert into country (ci_code, ci_name) values ("VI", "Virgils Islands, U.S.");
252 * insert into country (ci_code, ci_name) values ("VI", "Virgils and Futuna");
253 * insert into country (ci_code, ci_name) values ("VI", "Western Sahara");
254 * insert into country (ci_code, ci_name) values ("VI", "Yemen");
255 * insert into country (ci_code, ci_name) values ("ZM", "Zambia");
256 * insert into country (ci_code, ci_name) values ("ZM", "Zimbabwe");
257 * insert into country (ci_code, ci_name) values ("ZM", "Zimbabwe");
258
259 * SELECT COUNT(*) FROM country;
260
261 * SELECT * FROM country WHERE ci_code="VI";
262
263 * select * from country
264 where ci_name like "South";
265
--
-- Table structure for `country`
--
-- Table `country`
--   ci_code varchar(2) NOT NULL,
--   ci_name varchar(100) NOT NULL,
--   PRIMARY KEY (ci_code)
-- 
```

Hands on 2

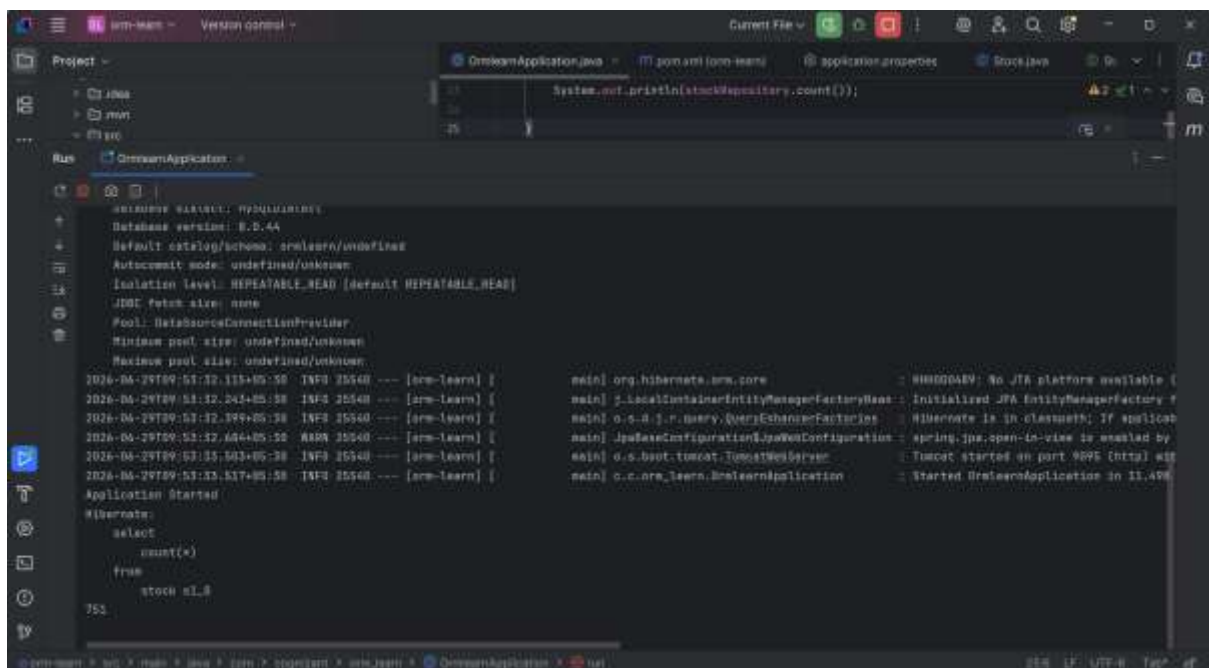
Write queries on stock table using Query Methods







OUTPUT:



DATABASE:

MySQL Workbench

Local instance MySQL56

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

- Info schema
- bankid
- course_mysql
- Tables
- Views
- Stored Procedures
- Functions
- event_management
- products
- Tables
- Views
- Stored Procedures
- Functions
- reports_db
- securitydb
- spring_loan
- studentdb
- sys

Administration Schema

Object Explorer

No object selected

File Edit View Query Database Server Tools Scripting Help

File 10 SQL File 12 SQL File 14 SQL File 15 SQL File 11 SQL File 12 SQL File 13 SQL File 14 stock_inserts.sql stock_inserts.sql

1 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-10', 100.00, 210.00, 1000000);

2 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-11', 102.00, 212.00, 1000000);

3 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-12', 104.00, 214.00, 1000000);

4 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-13', 106.00, 216.00, 1000000);

5 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-14', 108.00, 218.00, 1000000);

6 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-15', 110.00, 220.00, 1000000);

7 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-16', 112.00, 222.00, 1000000);

8 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-17', 114.00, 224.00, 1000000);

9 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-18', 116.00, 226.00, 1000000);

10 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-19', 118.00, 228.00, 1000000);

11 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-20', 120.00, 230.00, 1000000);

12 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-21', 122.00, 232.00, 1000000);

13 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-22', 124.00, 234.00, 1000000);

14 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-23', 126.00, 236.00, 1000000);

15 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-24', 128.00, 238.00, 1000000);

16 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-25', 130.00, 240.00, 1000000);

17 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-26', 132.00, 242.00, 1000000);

Result grid

#	st_code	st_date	st_open	st_close	st_volume
1	1000-10-10	2010-10-10	100.00	210.00	1000000
2	1000-10-11	2010-10-11	102.00	212.00	1000000
3	1000-10-12	2010-10-12	104.00	214.00	1000000
4	1000-10-13	2010-10-13	106.00	216.00	1000000
5	1000-10-14	2010-10-14	108.00	218.00	1000000

stock_inserts.sql

Output

Action Output

#	Time	Action	Message	Duration / Rows
1	09:45:52	Insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-10', 2010-10-10, 100.00, 210.00, 1000000);	1 record affected	0:00 sec
2	09:45:52	Insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-11', 2010-10-11, 102.00, 212.00, 1000000);	1 record affected	0:00 sec
3	09:45:52	Insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-12', 2010-10-12, 104.00, 214.00, 1000000);	1 record affected	0:00 sec

Automatic content help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

MySQL Workbench

Local instance MySQL56

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

- Info schema
- bankid
- course_mysql
- Tables
- Views
- Stored Procedures
- Functions
- event_management
- products
- Tables
- Views
- Stored Procedures
- Functions
- reports_db
- securitydb
- spring_loan
- studentdb
- sys

Administration Schema

Object Explorer

No object selected

File Edit View Query Database Server Tools Scripting Help

File 10 SQL File 12 SQL File 14 SQL File 15 SQL File 11 SQL File 12 SQL File 13 SQL File 14 stock_inserts.sql stock_inserts.sql

1 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-10', 100.00, 210.00, 1000000);

2 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-11', 102.00, 212.00, 1000000);

3 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-12', 104.00, 214.00, 1000000);

4 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-13', 106.00, 216.00, 1000000);

5 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-14', 108.00, 218.00, 1000000);

6 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-15', 110.00, 220.00, 1000000);

7 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-16', 112.00, 222.00, 1000000);

8 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-17', 114.00, 224.00, 1000000);

9 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-18', 116.00, 226.00, 1000000);

10 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-19', 118.00, 228.00, 1000000);

11 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-20', 120.00, 230.00, 1000000);

12 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-21', 122.00, 232.00, 1000000);

13 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-22', 124.00, 234.00, 1000000);

14 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-23', 126.00, 236.00, 1000000);

15 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-24', 128.00, 238.00, 1000000);

16 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-25', 130.00, 240.00, 1000000);

17 * insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-26', 132.00, 242.00, 1000000);

Result grid

#	st_code	st_date	st_open	st_close	st_volume
1	1000-10-10	2010-10-10	100.00	210.00	1000000
2	1000-10-11	2010-10-11	102.00	212.00	1000000
3	1000-10-12	2010-10-12	104.00	214.00	1000000
4	1000-10-13	2010-10-13	106.00	216.00	1000000
5	1000-10-14	2010-10-14	108.00	218.00	1000000

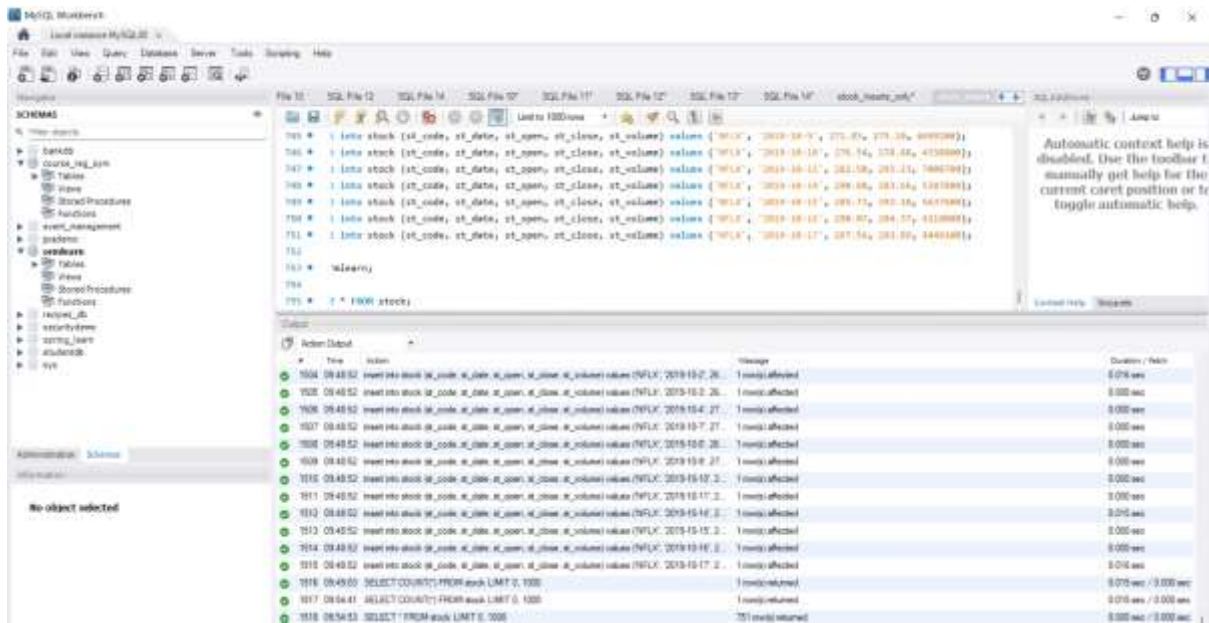
stock_inserts.sql

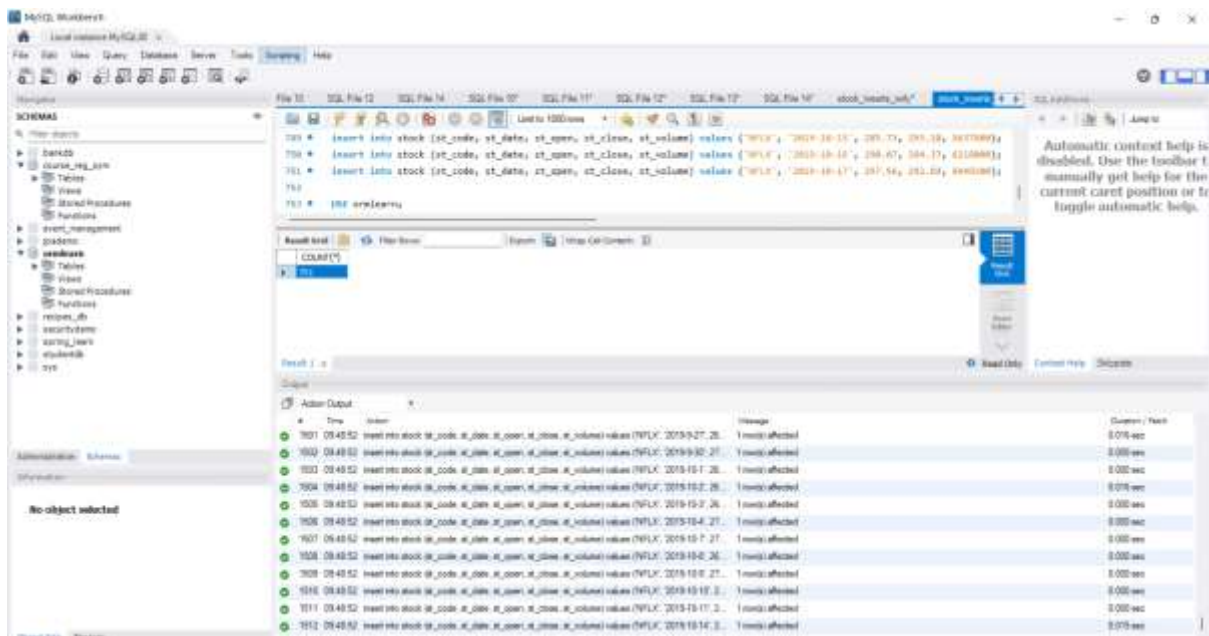
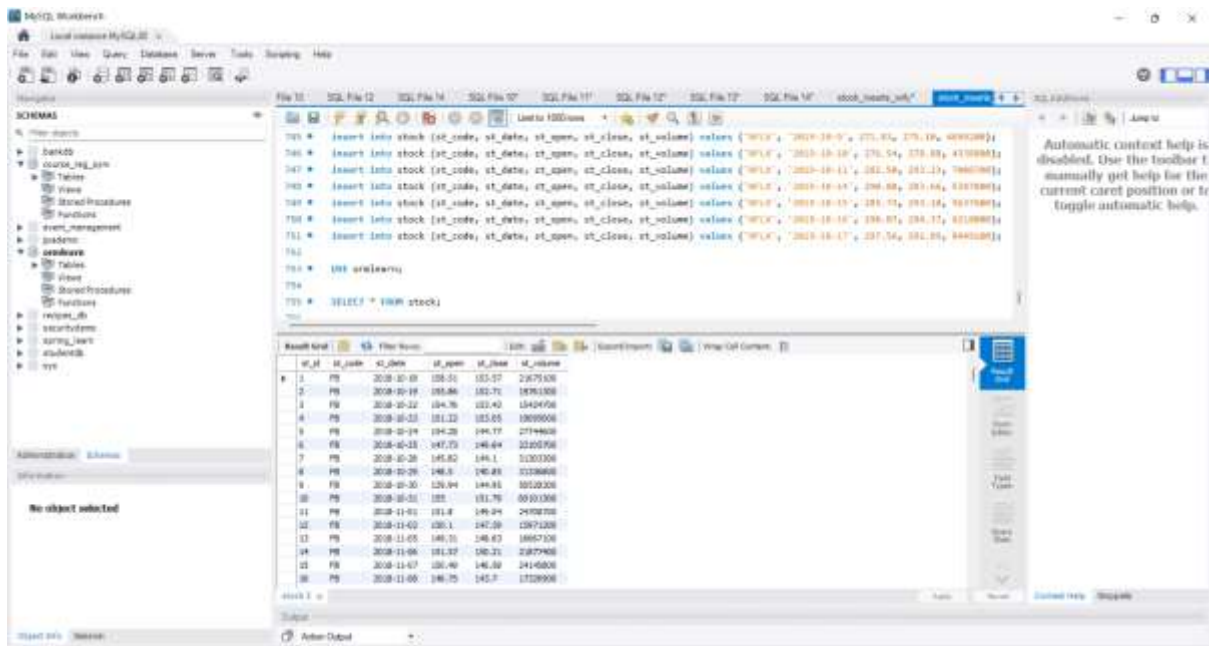
Output

Action Output

#	Time	Action	Message	Duration / Rows
1	09:45:52	Insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-10', 2010-10-10, 100.00, 210.00, 1000000);	1 record affected	0:00 sec
2	09:45:52	Insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-11', 2010-10-11, 102.00, 212.00, 1000000);	1 record affected	0:00 sec
3	09:45:52	Insert into stock (st_code, st_date, st_open, st_close, st_volume) values ('1000-10-12', 2010-10-12, 104.00, 214.00, 1000000);	1 record affected	0:00 sec

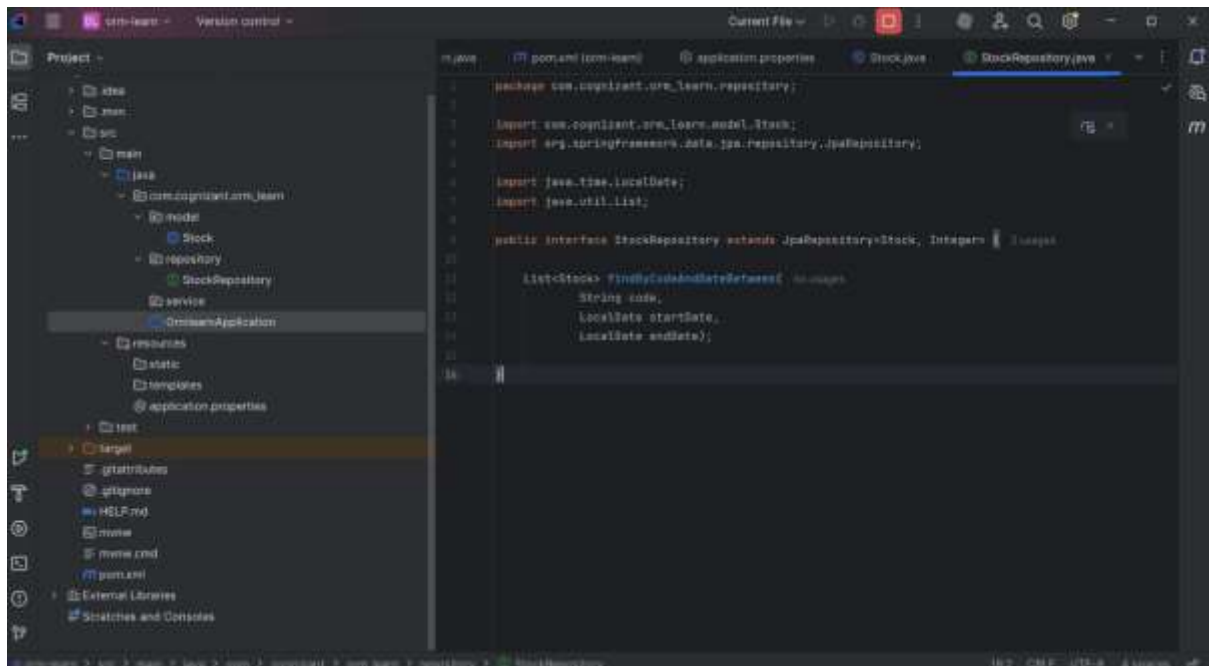
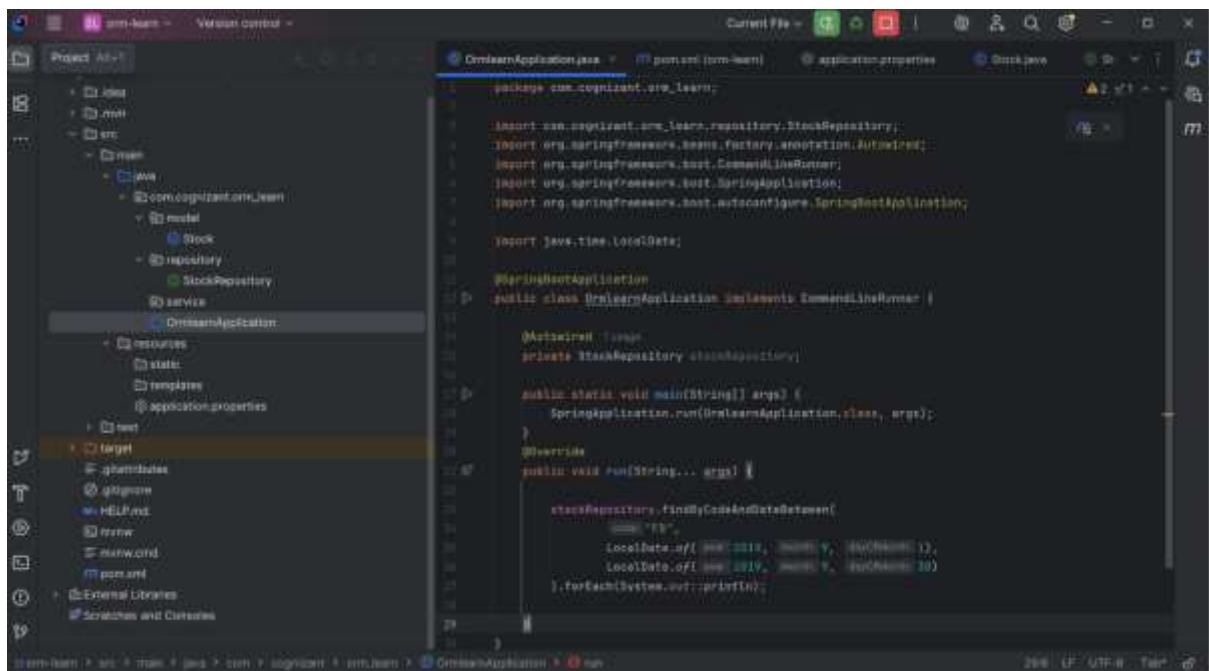
Automatic content help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.





Query Methods required for the following scenarios

- Get all stock details of Facebook in the month of September 2019. Expected data result of Query Method below.



OUTPUT:

```

Project: ...
Run: ...
2020-06-29T10:04:41.917+05:30 INFO 25924 --- [main] c.s.koot.TomcatTomcatWebServer : Tomcat started on port 9095 (http) wit
2020-06-29T10:04:41.922+05:30 INFO 25924 --- [main] c.s.crm.Learn.BrowserApplication : Started BrowserApplication in 11.81s
Hibernate:
select
  sl_b.st_id,
  sl_b.st_class,
  sl_b.st_code,
  sl_b.st_date,
  sl_b.st_open,
  sl_b.st_volume
from
  stock sl_b
where
  sl_b.st_code=?
  and sl_b.st_date between ? and ?
Stock[id=218, code='FB', date='2019-09-03', open=184.0, close=182.39, volume=9779480]
Stock[id=219, code='FB', date='2019-09-04', open=184.45, close=187.14, volume=11308000]
Stock[id=220, code='FB', date='2019-09-05', open=188.53, close=190.9, volume=13076700]
Stock[id=221, code='FB', date='2019-09-06', open=190.21, close=187.49, volume=15124800]
Stock[id=222, code='FB', date='2019-09-09', open=187.73, close=188.76, volume=14722400]
Stock[id=223, code='FB', date='2019-09-10', open=187.44, close=186.17, volume=15455900]
Stock[id=224, code='FB', date='2019-09-11', open=186.44, close=188.49, volume=11761700]
Stock[id=225, code='FB', date='2019-09-12', open=189.86, close=187.47, volume=11419800]
Stock[id=226, code='FB', date='2019-09-13', open=187.33, close=187.19, volume=11461100]

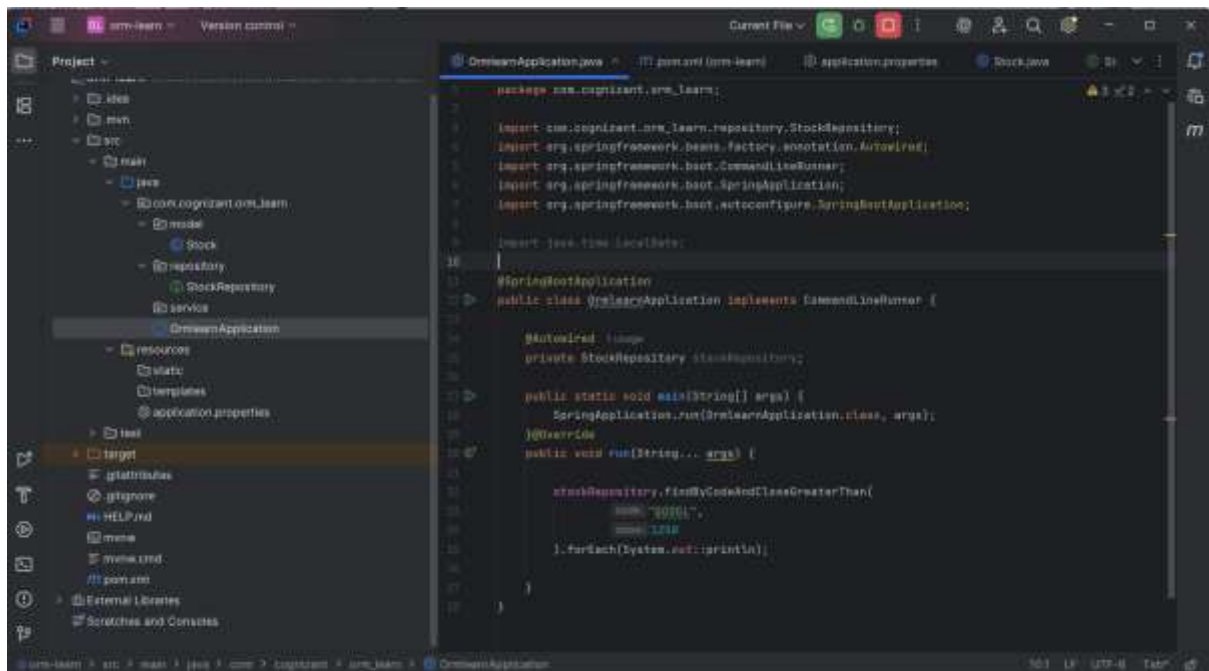
```

```

Project: ...
Run: ...
where
  sl_b.st_code=?
  and sl_b.st_date between ? and ?
Stock[id=218, code='FB', date='2019-09-03', open=184.0, close=182.39, volume=9779480]
Stock[id=219, code='FB', date='2019-09-04', open=184.45, close=187.14, volume=11308000]
Stock[id=220, code='FB', date='2019-09-05', open=188.53, close=190.9, volume=13076700]
Stock[id=221, code='FB', date='2019-09-06', open=190.21, close=187.49, volume=15124800]
Stock[id=222, code='FB', date='2019-09-09', open=187.73, close=188.76, volume=14722400]
Stock[id=223, code='FB', date='2019-09-10', open=187.44, close=186.17, volume=15455900]
Stock[id=224, code='FB', date='2019-09-11', open=186.44, close=188.49, volume=11761700]
Stock[id=225, code='FB', date='2019-09-12', open=189.86, close=187.47, volume=11419800]
Stock[id=226, code='FB', date='2019-09-13', open=187.33, close=187.19, volume=11461100]
Stock[id=227, code='FB', date='2019-09-16', open=186.93, close=186.22, volume=8444800]
Stock[id=228, code='FB', date='2019-09-17', open=186.66, close=188.08, volume=9671100]
Stock[id=229, code='FB', date='2019-09-18', open=188.09, close=188.14, volume=9681900]
Stock[id=230, code='FB', date='2019-09-19', open=188.66, close=189.14, volume=10192700]
Stock[id=231, code='FB', date='2019-09-20', open=190.66, close=189.93, volume=19434200]
Stock[id=232, code='FB', date='2019-09-23', open=189.34, close=186.80, volume=13327600]
Stock[id=233, code='FB', date='2019-09-24', open=187.98, close=181.28, volume=18546000]
Stock[id=234, code='FB', date='2019-09-25', open=181.45, close=181.8, volume=12068100]
Stock[id=235, code='FB', date='2019-09-26', open=181.15, close=180.11, volume=10881300]
Stock[id=236, code='FB', date='2019-09-27', open=180.49, close=177.1, volume=1466200]
Stock[id=237, code='FB', date='2019-09-30', open=177.87, close=176.53, volume=10748000]

```

- Get all google stock details where the stock price was greater than 1250



```
package org.cognizant.orm.learn;

import org.cognizant.orm.learn.repository.StockRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

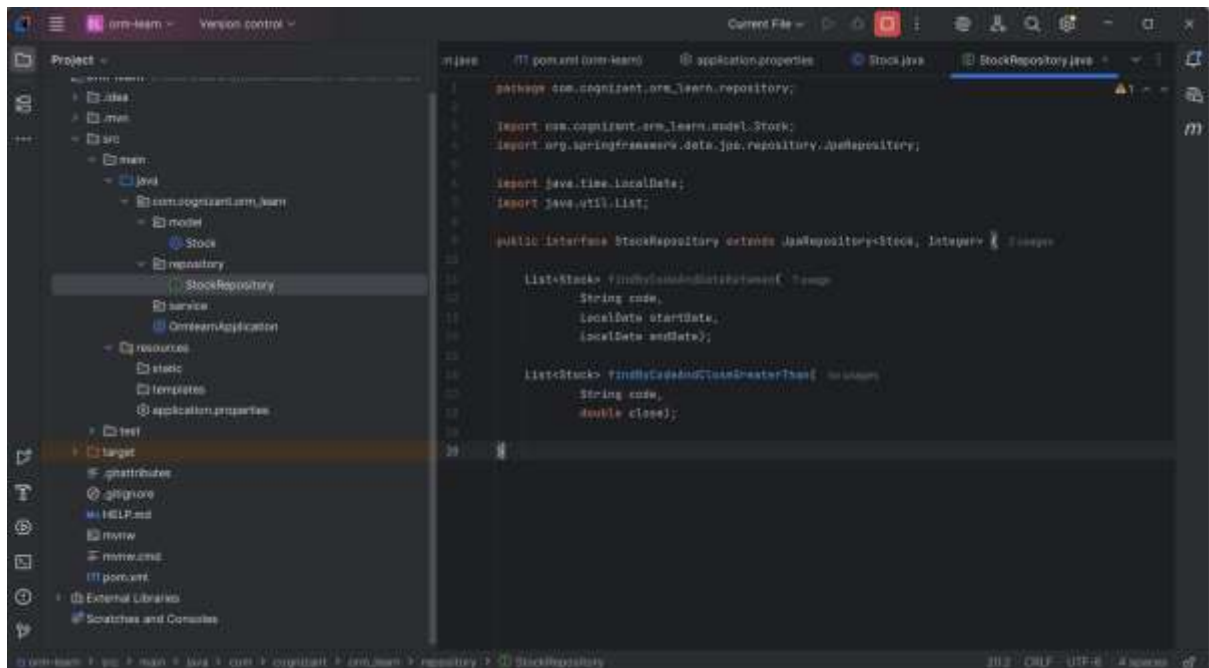
import java.time.LocalDate;

@SpringBootApplication
public class OrmLearnApplication implements CommandLineRunner {

    @Autowired
    private StockRepository stockRepository;

    public static void main(String[] args) {
        SpringApplication.run(OrmLearnApplication.class, args);
    }

    @Override
    public void run(String... args) {
        stockRepository.findByCodeAndDateGreaterThan(
            "10101",
            LocalDate.now().minusDays(100)
        ).forEach(System.out::println);
    }
}
```



```
package org.cognizant.orm.learn.repository;

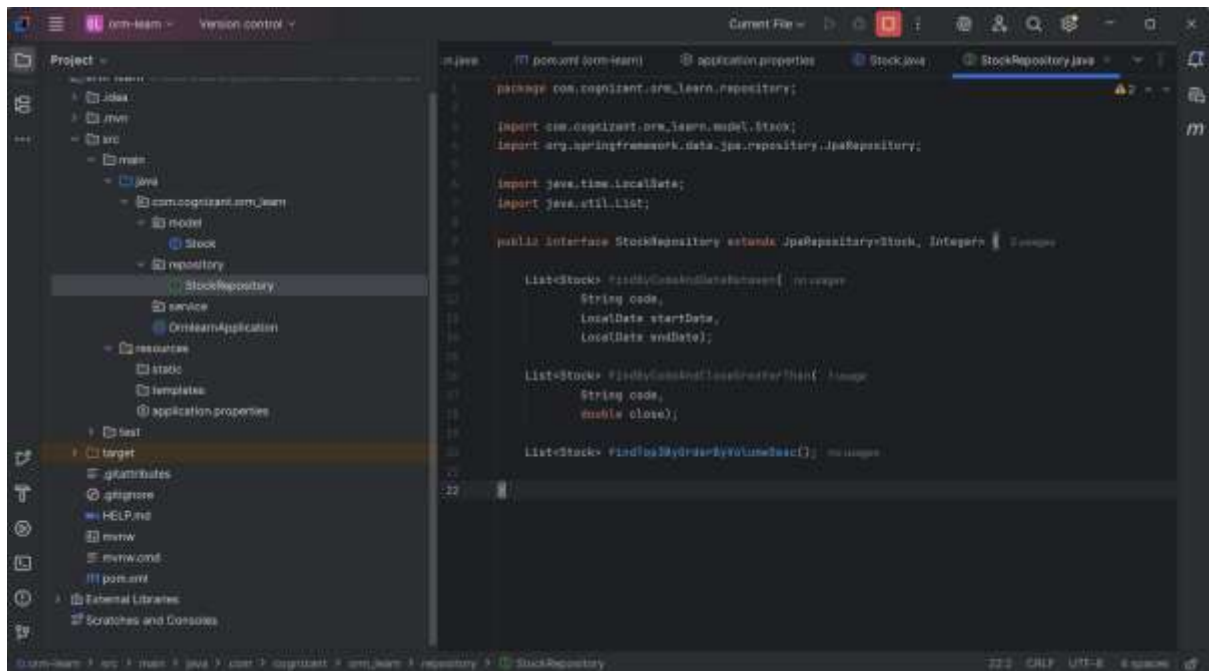
import org.cognizant.orm.learn.model.Stock;
import org.springframework.data.jpa.repository.JpaRepository;
import java.time.LocalDate;
import java.util.List;

public interface StockRepository extends JpaRepository<Stock, Integer> {

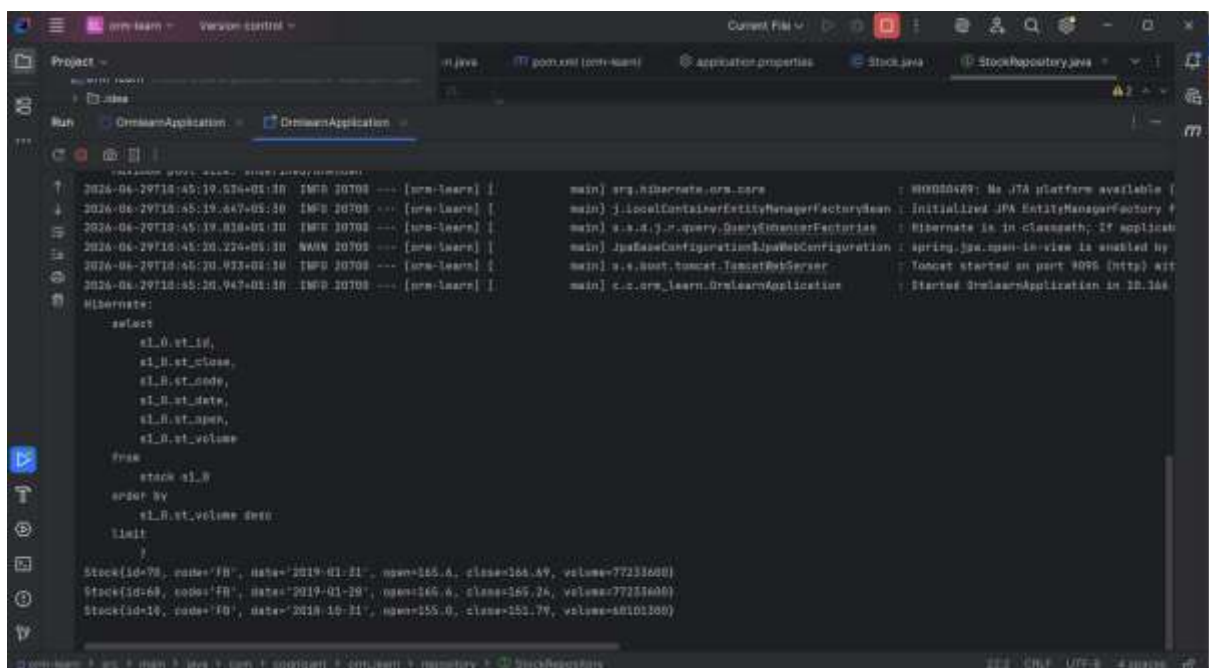
    List<Stock> findByCodeAndDateBetween(
        String code,
        LocalDate startDate,
        LocalDate endDate);

    List<Stock> findByCodeAndDateGreaterThan(
        String code,
        double date);
}
```

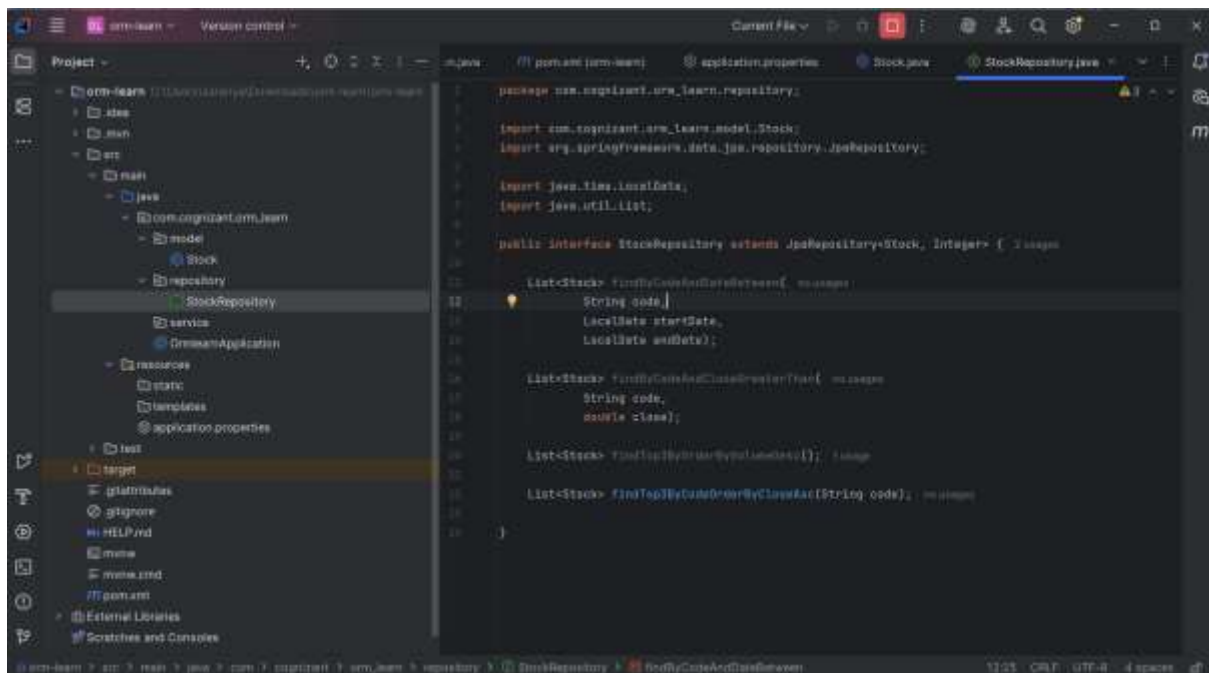
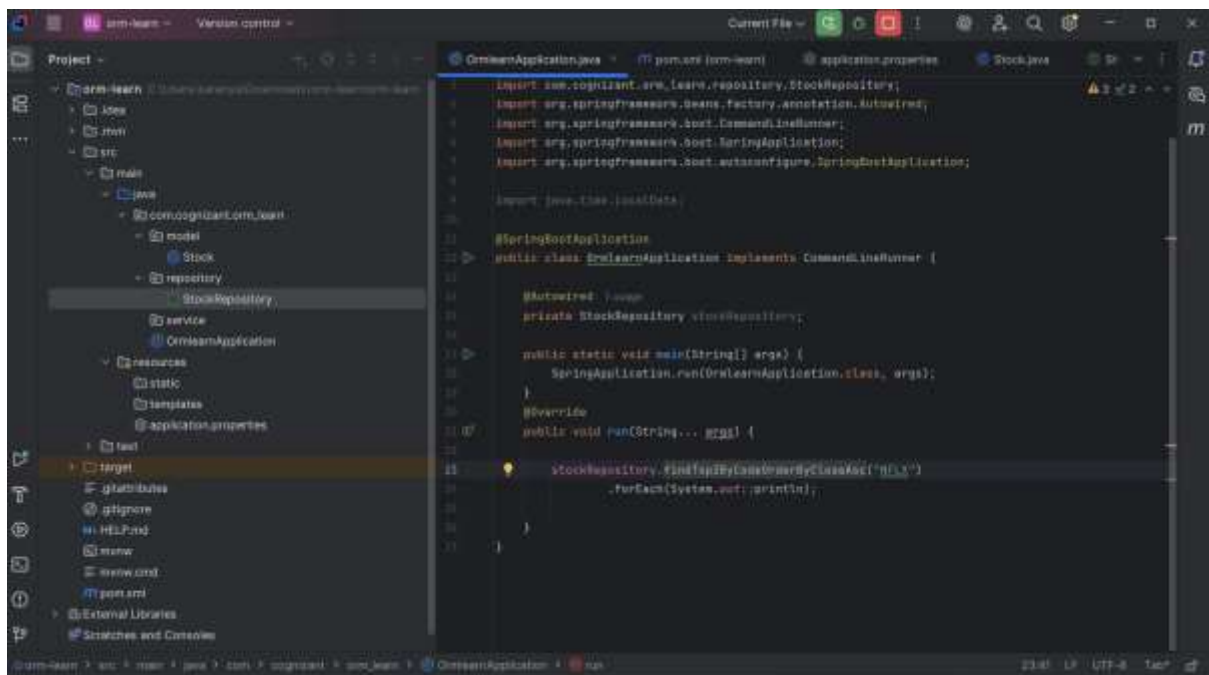
OUTPUT:



OUTPUT:



- Identify three dates when Netflix stocks were the lowest



OUTPUT:

The screenshot shows an IDE with a project named 'orm-learn'. The console displays a SQL query executed by Hibernate:

```
select
  sl_0.st_id,
  sl_0.st_class,
  sl_0.st_code,
  sl_0.st_date,
  sl_0.st_open,
  sl_0.st_volume
from
  stock sl_0
where
  sl_0.st_code=?
order by
  sl_0.st_class
limit
  ?
```

The results of the query are shown below:

```
Stock(id=544, code='NFLX', date='2018-12-24', open=242.9, close=233.88, volume=9547688)
Stock(id=544, code='NFLX', date='2018-12-20', open=239.98, close=236.77, volume=13882708)
Stock(id=541, code='NFLX', date='2018-12-17', open=244.78, close=240.0, volume=10181888)
```

The Java application 'OrmLearnApplication' is running, and the console shows the following output:

```
main] c.s.d.j.c.query.QueryEnhancerFactory : Hibernate is in classpath; If applicat
main] JpaBaseConfiguration$JpaWebConfiguration : spring-jpa.open-in-view is enabled by
main] c.s.root.test.TestWebServer : Test started on port 9004 (http) wit
main] c.s.orm.learn.OrmLearnApplication : Started OrmLearnApplication in 10.57s
```

Demonstrate implementation of O/R Mapping

Hands on 3

Create payroll tables and bean mapping

The screenshot shows the project structure of 'orm-learn' in the IDE. The project is organized into several packages:

- com.cognizant.orm.learn
 - model
 - Department
 - Employee
 - Skill
 - repository
 - DepartmentRepository
 - EmployeeRepository
 - SkillRepository
 - service
 - DepartmentService
 - EmployeeService
 - SkillService
 - OrmLearnApplication
- resources
 - static
 - templates
- application.properties
- test
- target
- gitattributes

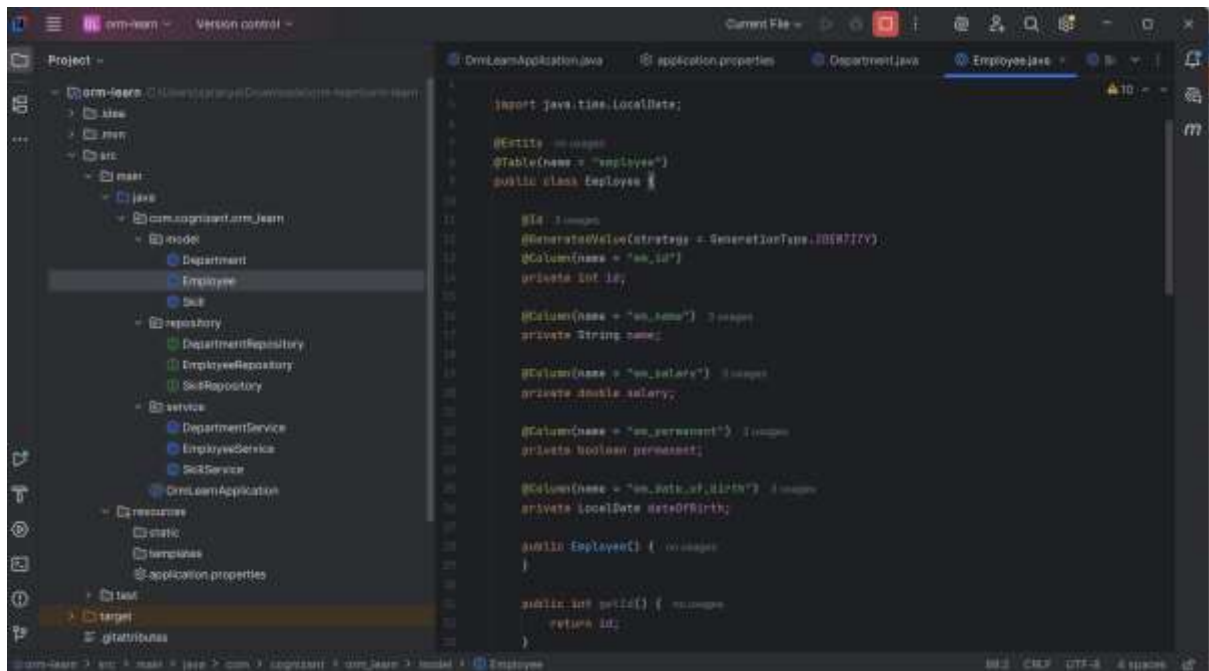
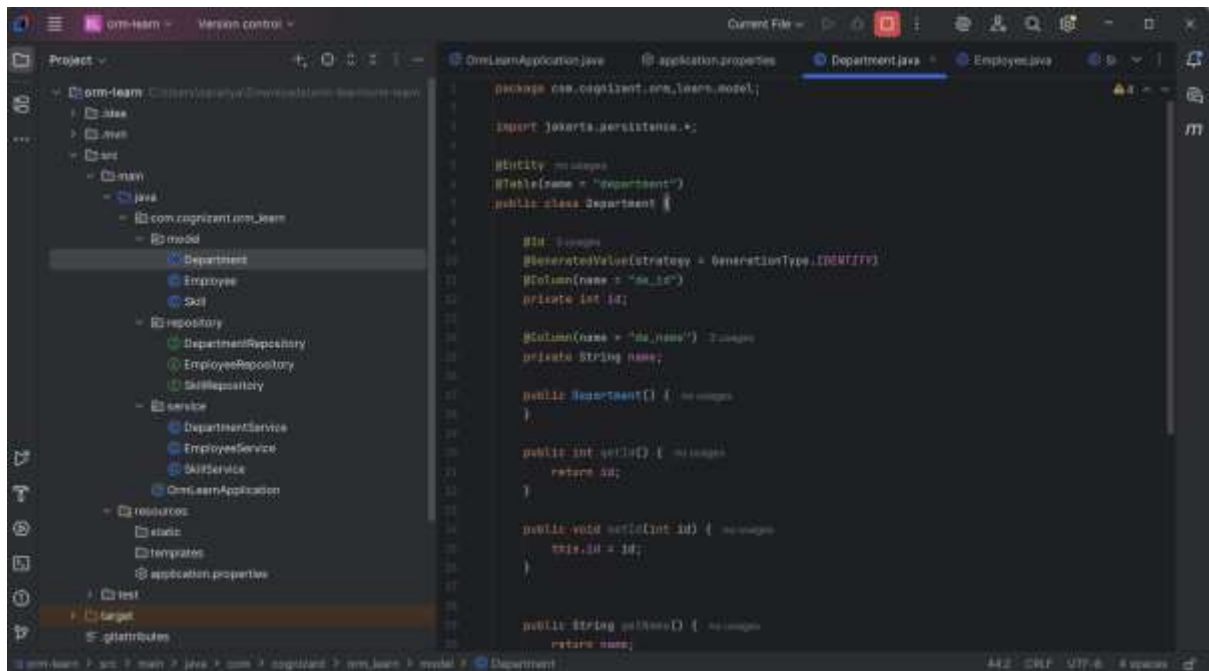
The 'OrmLearnApplication.java' file is open, showing the following code:

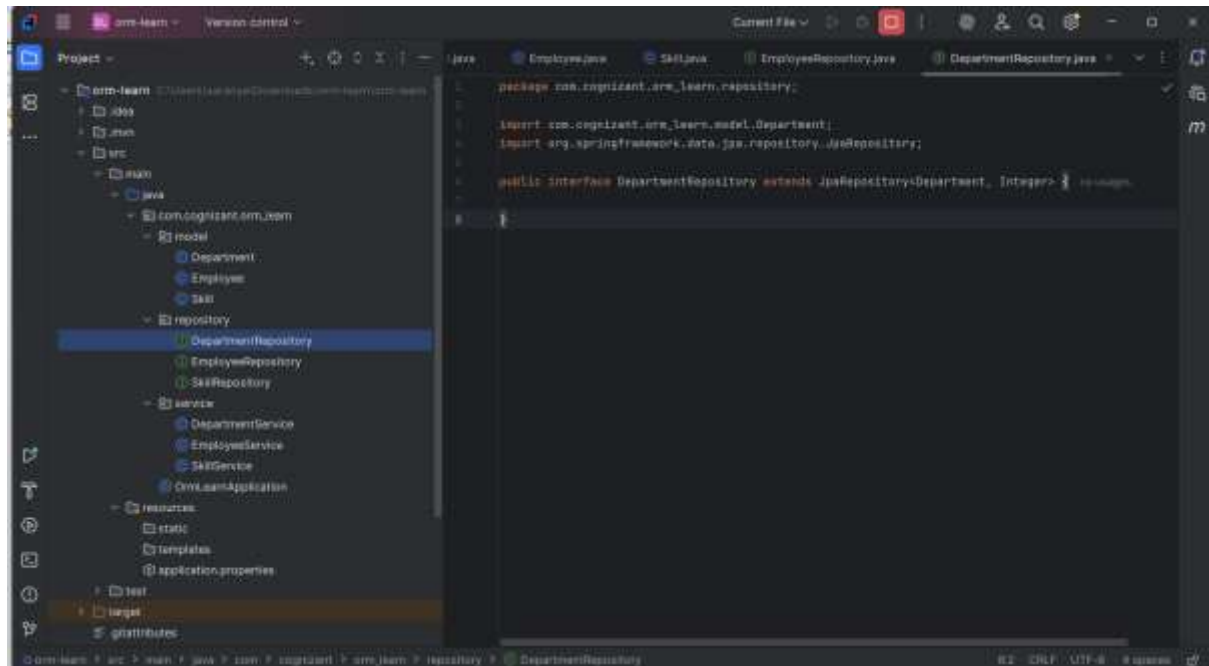
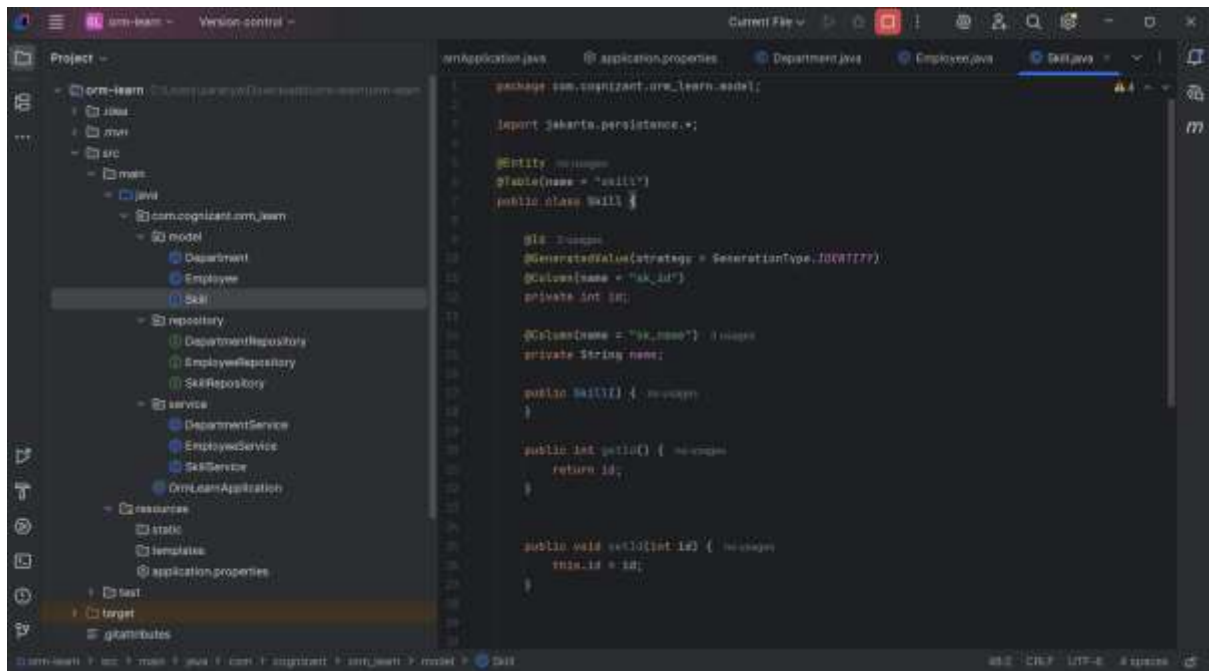
```
package com.cognizant.orm.learn;

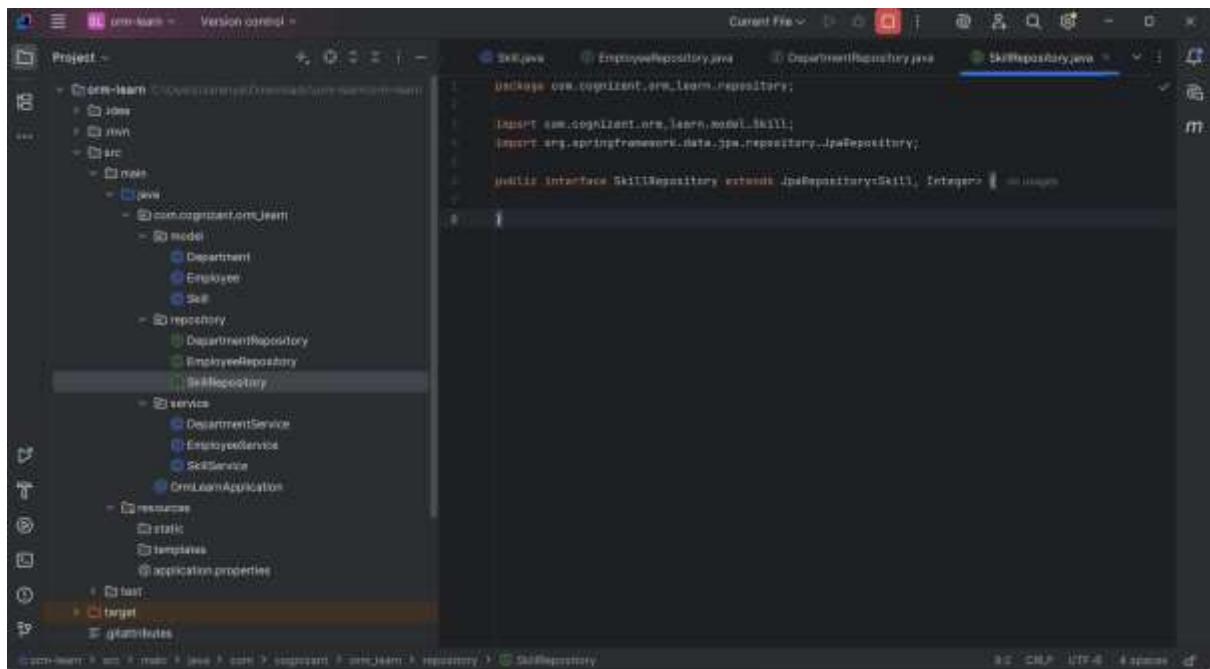
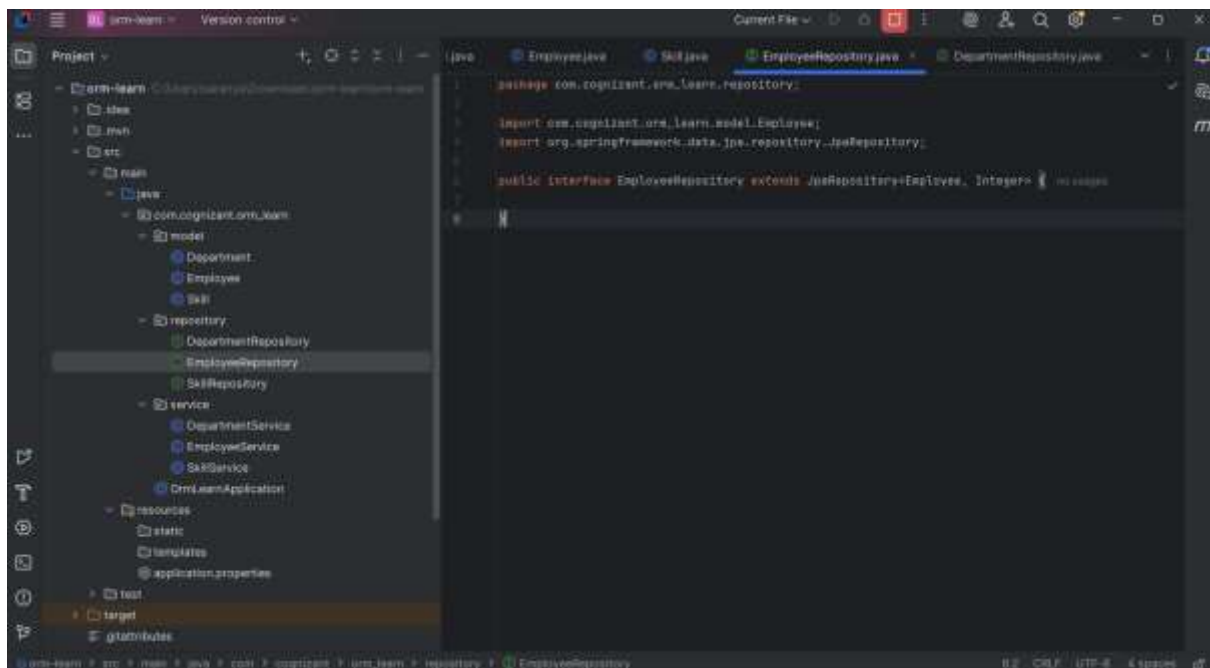
import org.springframework.boot.SpringApplication;

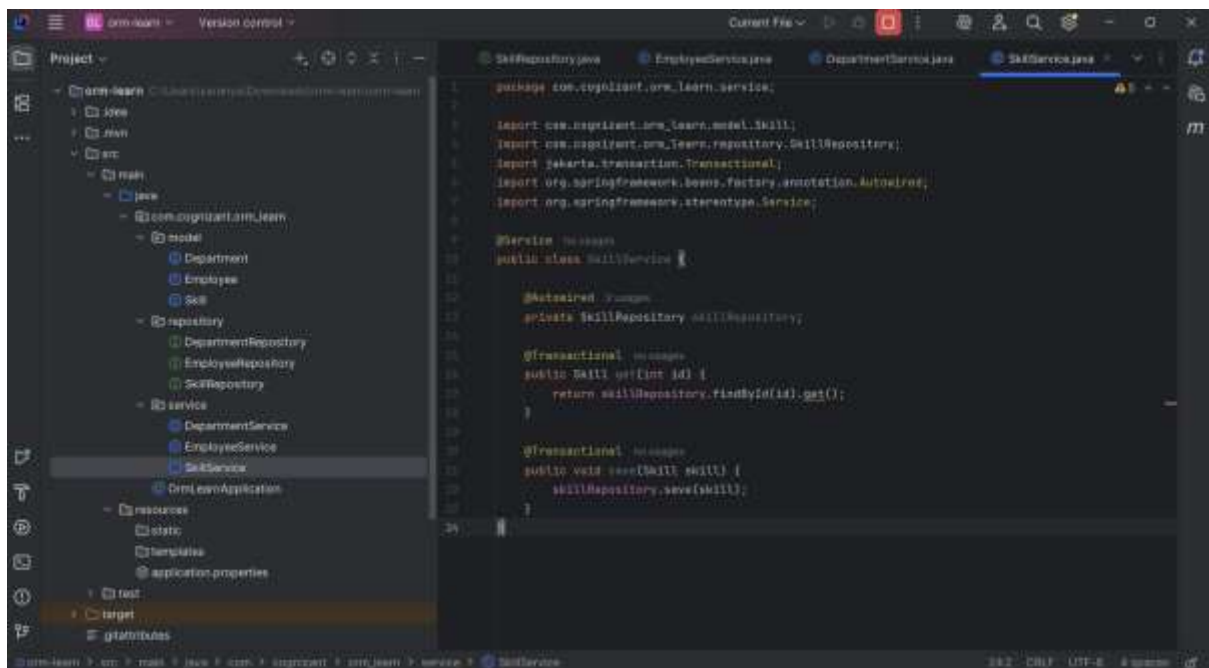
public class OrmLearnApplication {

    public static void main(String[] args) {
        SpringApplication.run(OrmLearnApplication.class, args);
    }
}
```

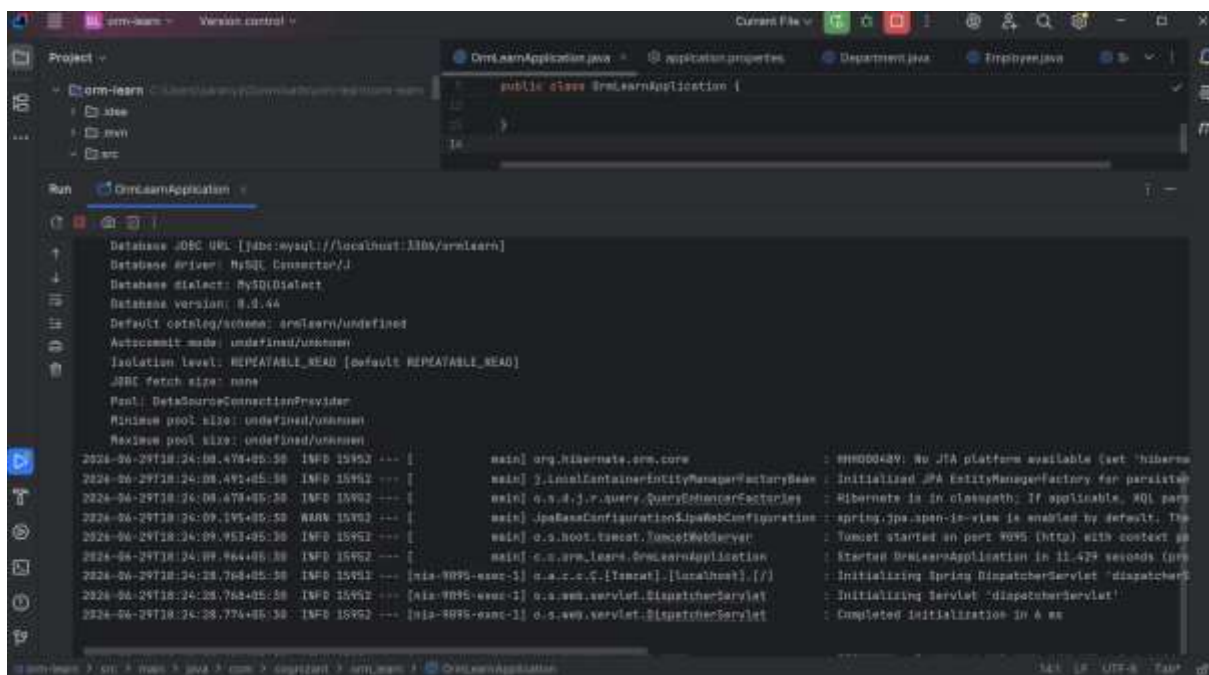




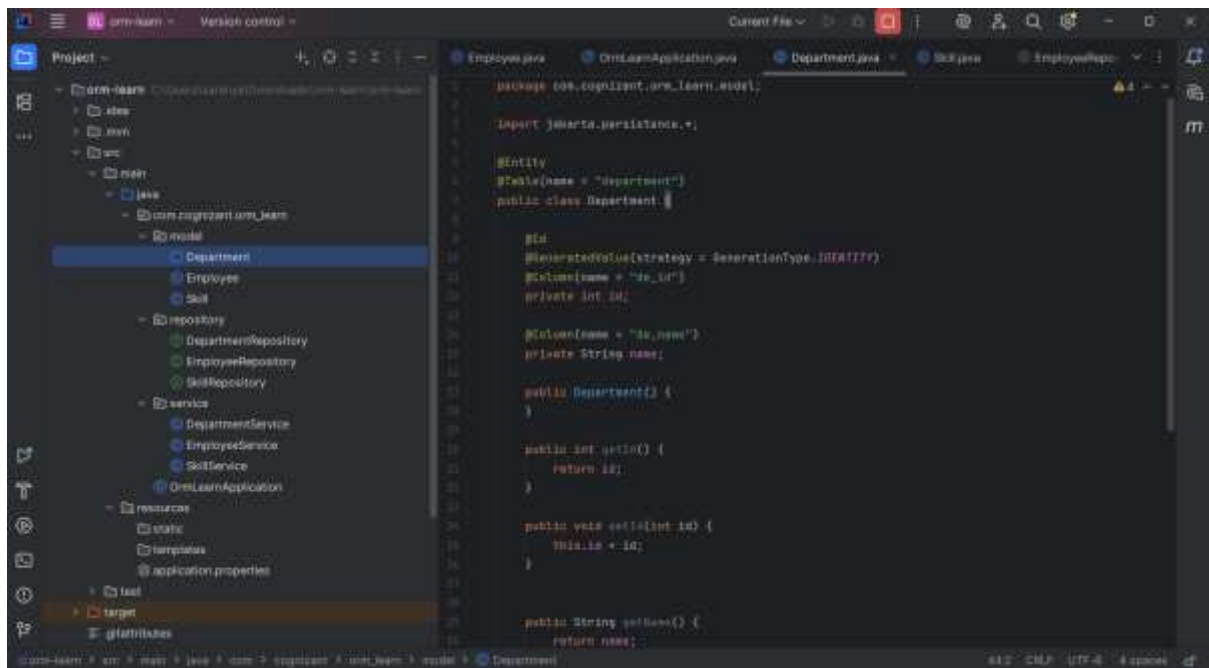
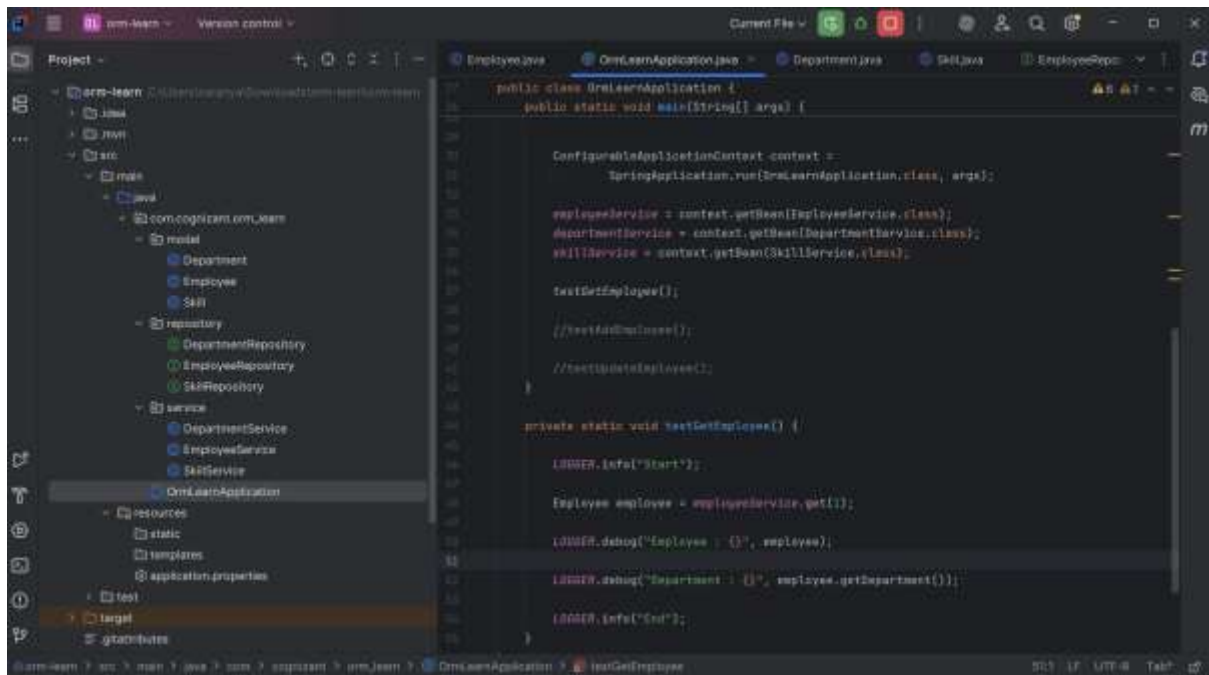


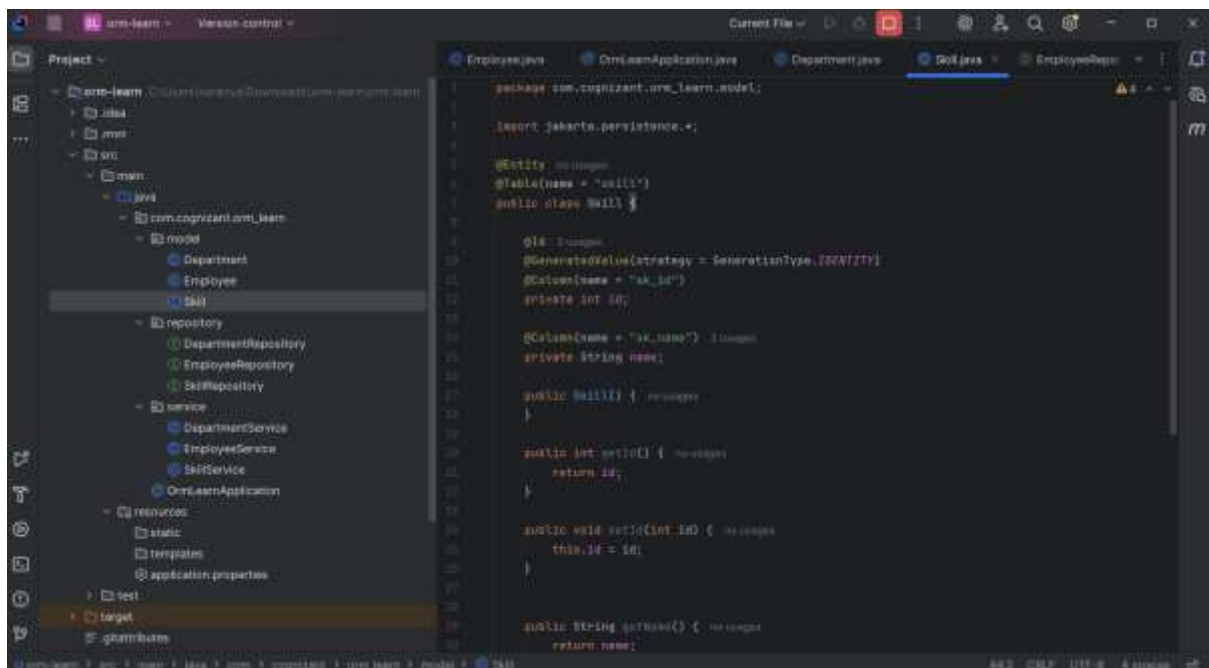


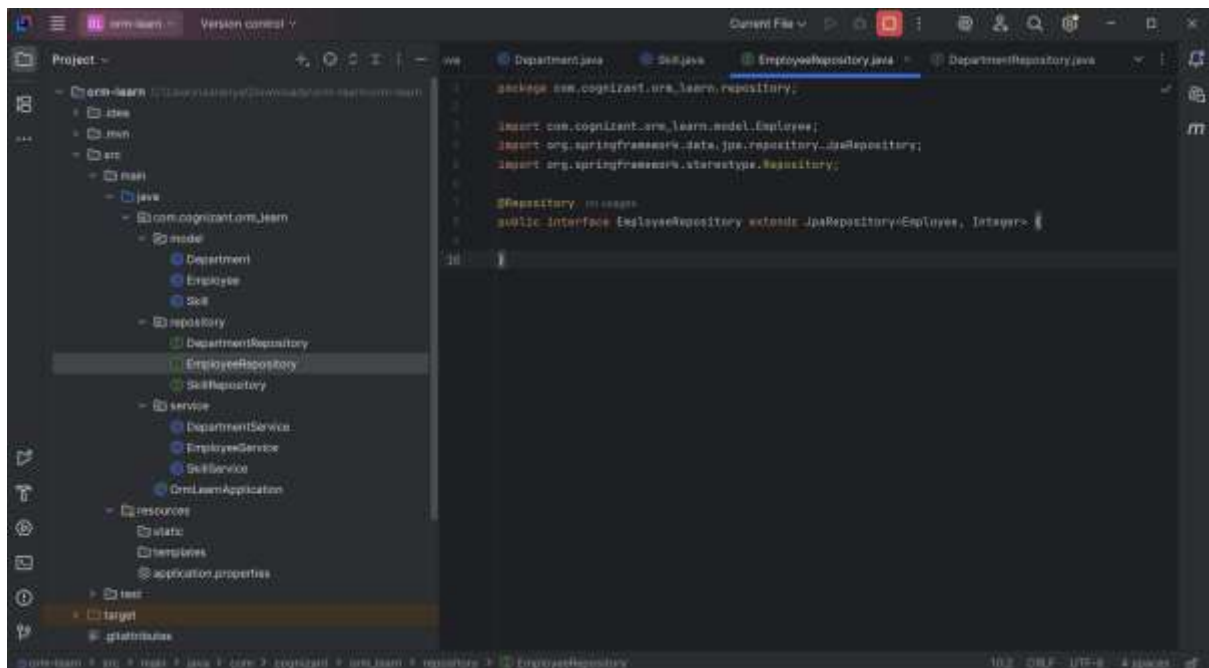
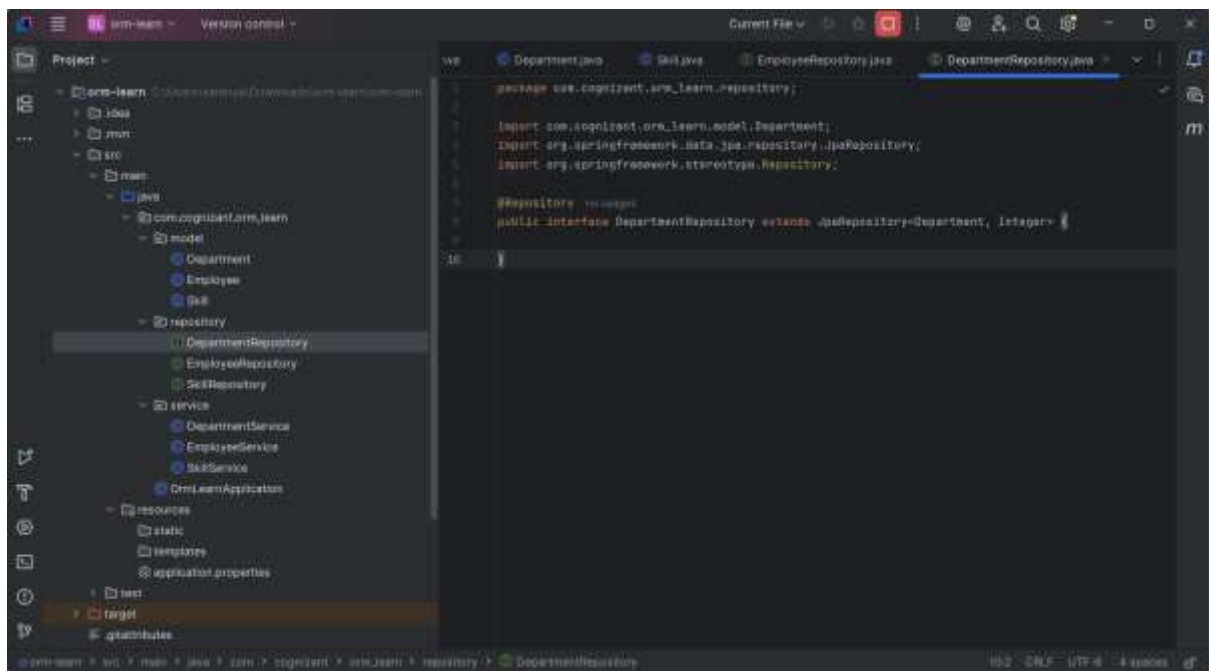
OUTPUT:

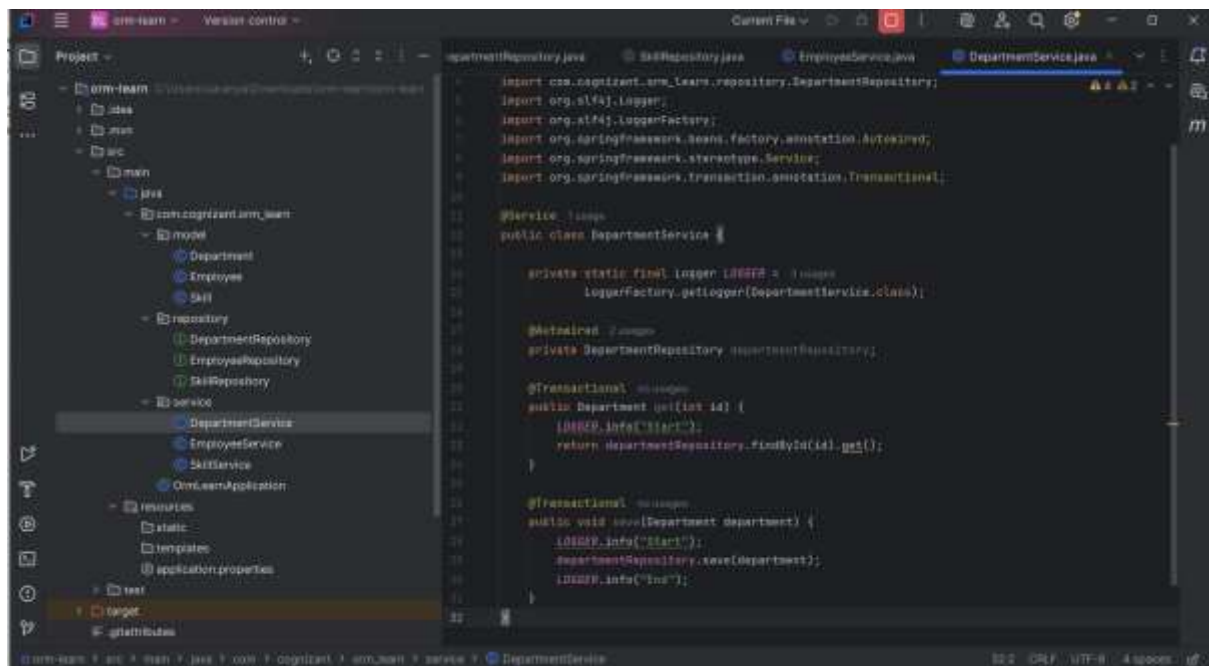
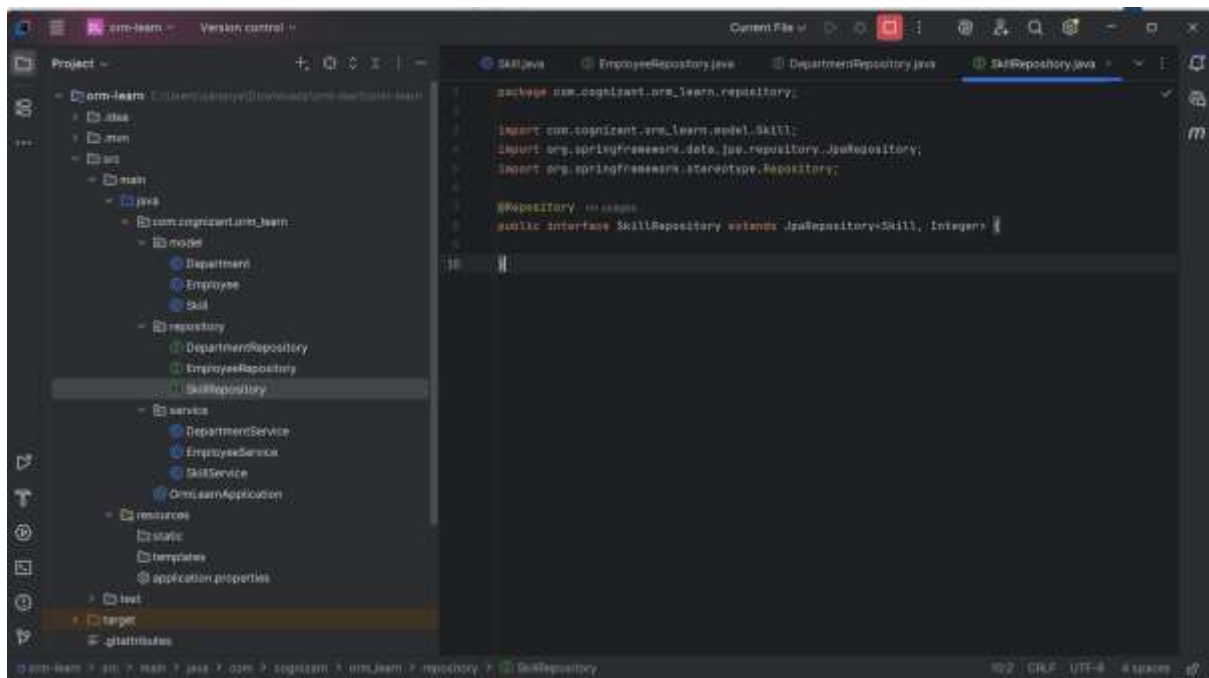


Getting Employee along with Department



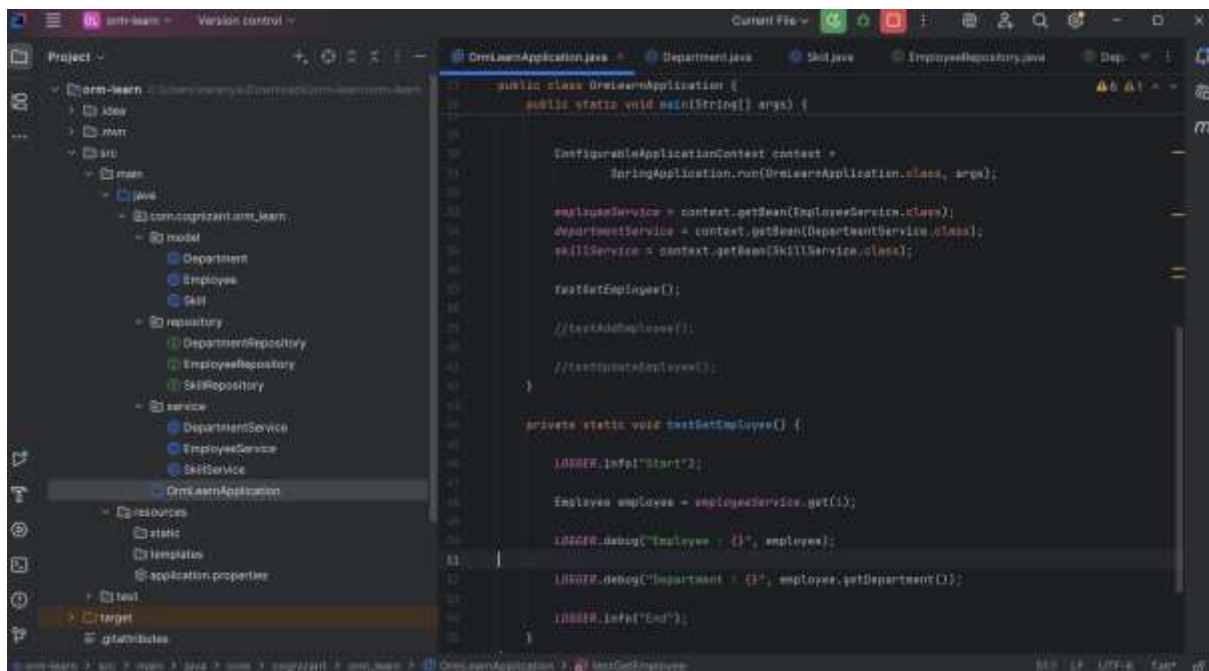




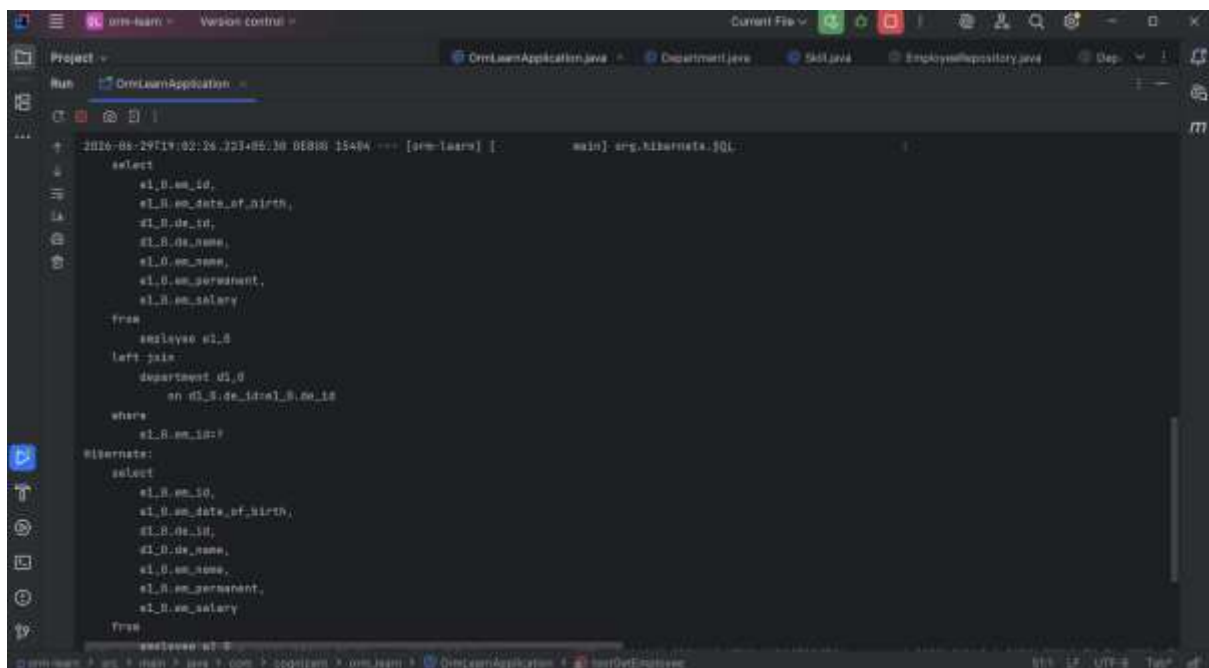


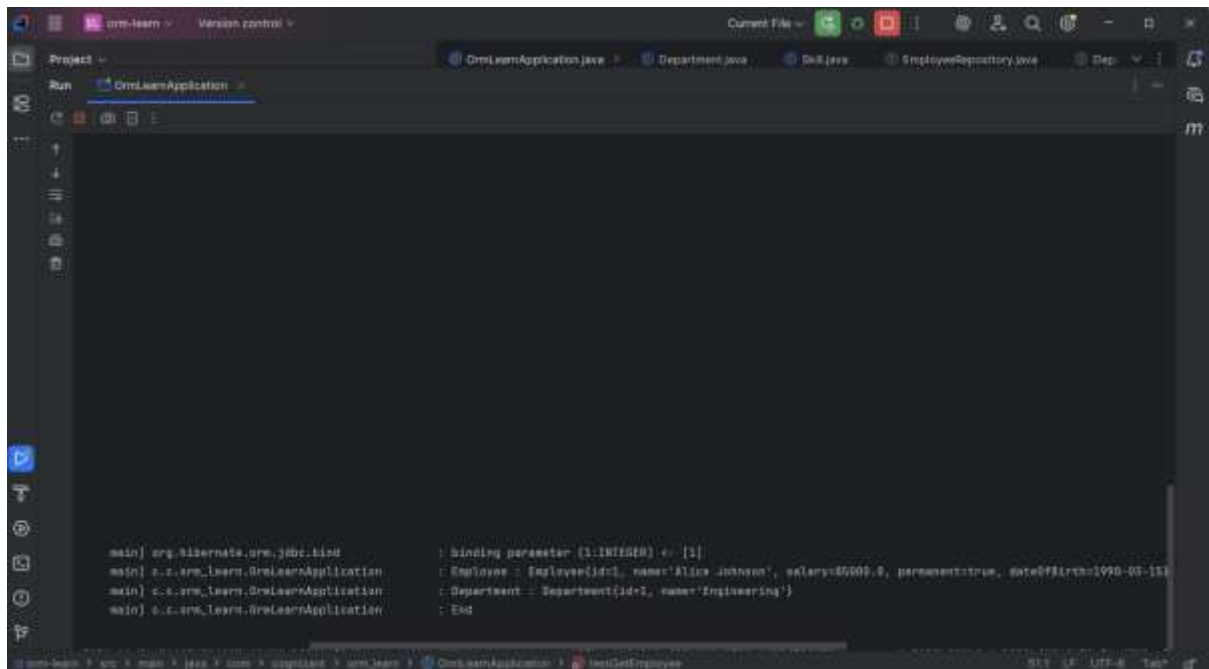
```
1 import org.springframework.stereotype.Service;
2 import org.springframework.transaction.annotation.Transactional;
3
4 @Service
5 public class EmployeeService {
6
7     private static final Logger LOGGER = Logger.getLogger(EmployeeService.class);
8
9     @Autowired
10     private EmployeeRepository employeeRepository;
11
12     @Transactional
13     public Employee getById(int id) {
14         LOGGER.info("Start");
15         return employeeRepository.findById(id).get();
16     }
17
18     @Transactional
19     public void save(Employee employee) {
20         LOGGER.info("Start");
21         employeeRepository.save(employee);
22         LOGGER.info("End");
23     }
24 }
```

```
1 import com.cognizant.orn_learn.repository.SkillRepository;
2 import org.slf4j.Logger;
3 import org.slf4j.LoggerFactory;
4 import org.springframework.beans.factory.annotation.Autowired;
5 import org.springframework.stereotype.Service;
6 import org.springframework.transaction.annotation.Transactional;
7
8 @Service
9 public class SkillService {
10
11     private static final Logger LOGGER = Logger.getLogger(SkillService.class);
12
13     @Autowired
14     private SkillRepository skillRepository;
15
16     @Transactional
17     public Skill getById(int id) {
18         LOGGER.info("Start");
19         return skillRepository.findById(id).get();
20     }
21
22     @Transactional
23     public void save(Skill skill) {
24         LOGGER.info("Start");
25         skillRepository.save(skill);
26         LOGGER.info("End");
27     }
28 }
```

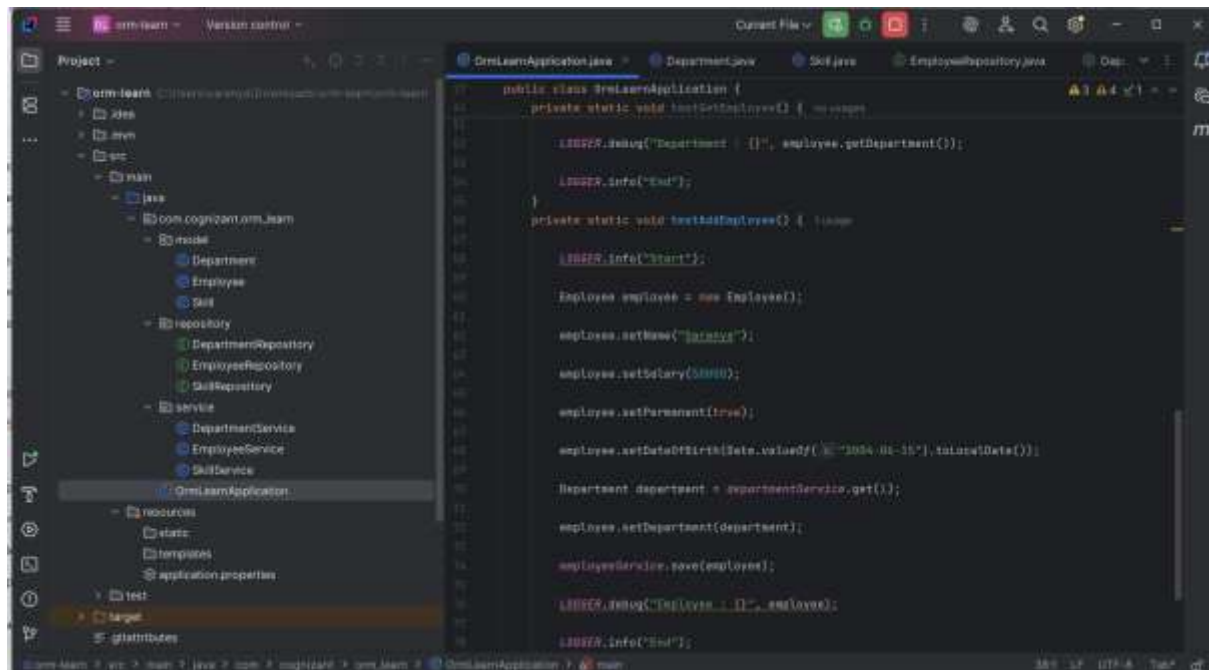


OUTPUT:





Add Employee



```

Project -
  - orm-learn
    - src
      - main
        - java
          - org
            - orm
              - learn
                - OmlLearnApplication.java
                - Department.java
                - Skill.java
                - EmployeeRepository.java
                - Dep...

Run - OmlLearnApplication -
  2024-06-29T19:12:04.143+05:30 DEBUG 23304 --- [orm-learn] [main] org.hibernate.SQL
  select
    dl_0.de_id,
    dl_0.de_name
  from
    department dl_0
  where
    dl_0.de_id=?
  Hibernate:
  select
    dl_0.de_id,
    dl_0.de_name
  from
    department dl_0
  where
    dl_0.de_id=?
  2024-06-29T19:12:04.144+05:30 TRACE 23304 --- [orm-learn] [main] org.hibernate.orm.jdbc.bind
  2024-06-29T19:12:04.239+05:30 INFO 23304 --- [orm-learn] [main] c.o.orm.learn.service.EmployeeService
  2024-06-29T19:12:04.337+05:30 DEBUG 23304 --- [orm-learn] [main] org.hibernate.SQL
  insert
  into
  
```

```

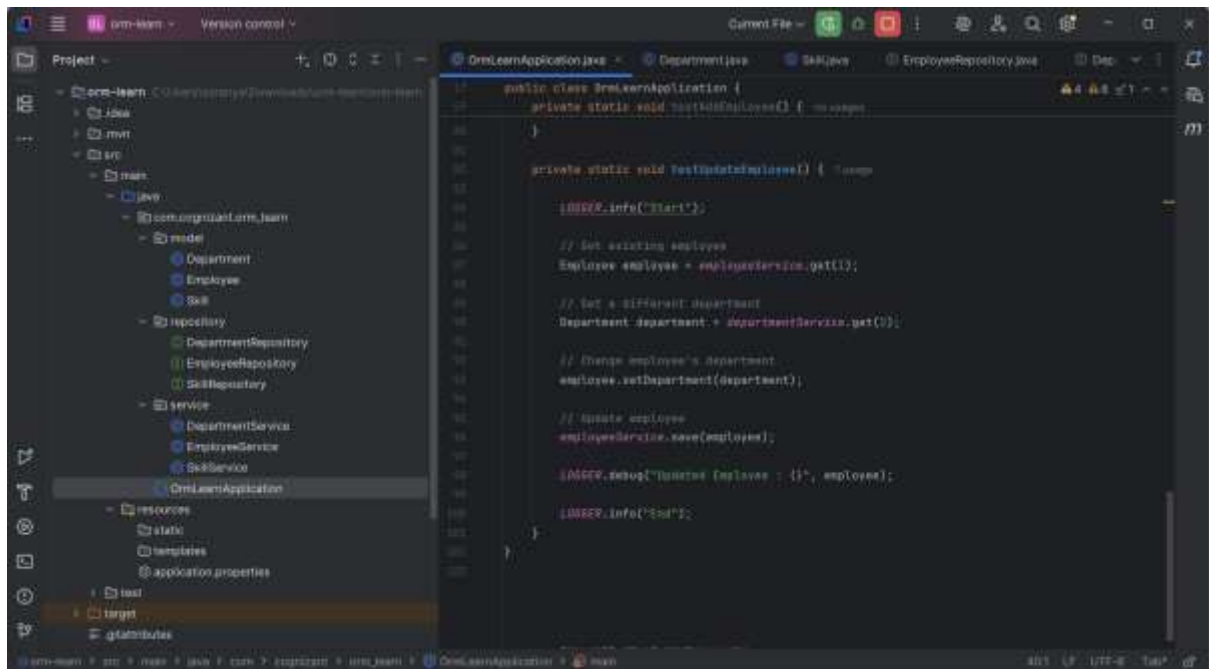
Project -
  - orm-learn
    - src
      - main
        - java
          - org
            - orm
              - learn
                - OmlLearnApplication.java
                - Department.java
                - Skill.java
                - EmployeeRepository.java
                - Dep...

Run - OmlLearnApplication -
  org)
  org)

  main] org.hibernate.orm.jdbc.bind
  main] org.hibernate.orm.jdbc.bind
  main] org.hibernate.orm.jdbc.bind
  main] org.hibernate.orm.jdbc.bind
  main] org.hibernate.orm.jdbc.bind
  main] c.o.orm.learn.service.EmployeeService
  main] c.o.orm.learn.OmlLearnApplication
  main] c.o.orm.learn.OmlLearnApplication

  : binding parameter (1:DATE) <- [2004-06-15]
  : binding parameter (2:INTEGER) <- [1]
  : binding parameter (3:VARCHAR) <- [Saranya]
  : binding parameter (4:BOOLEAN) <- [true]
  : binding parameter (5:DOUBLE) <- [50000.0]
  : End
  : Employee { Employee(id=14, name='Saranya', salary=50000.0, permanent=true, dateOfBirth=2004-06-15)
  : End
  
```

Update Employee



OUTPUT:

The image displays two screenshots of an IDE, likely IntelliJ IDEA, showing the development of a Java application that interacts with a database. The top screenshot shows a SQL query being executed, and the bottom screenshot shows the corresponding Java code.

Top Screenshot: SQL Query

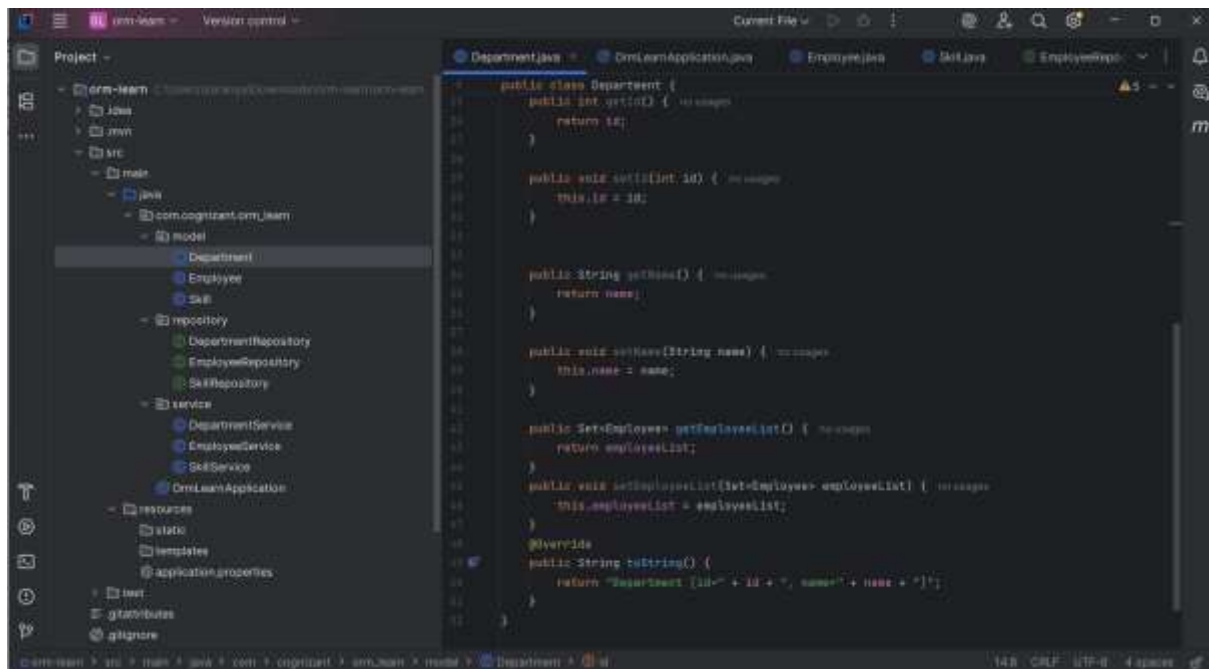
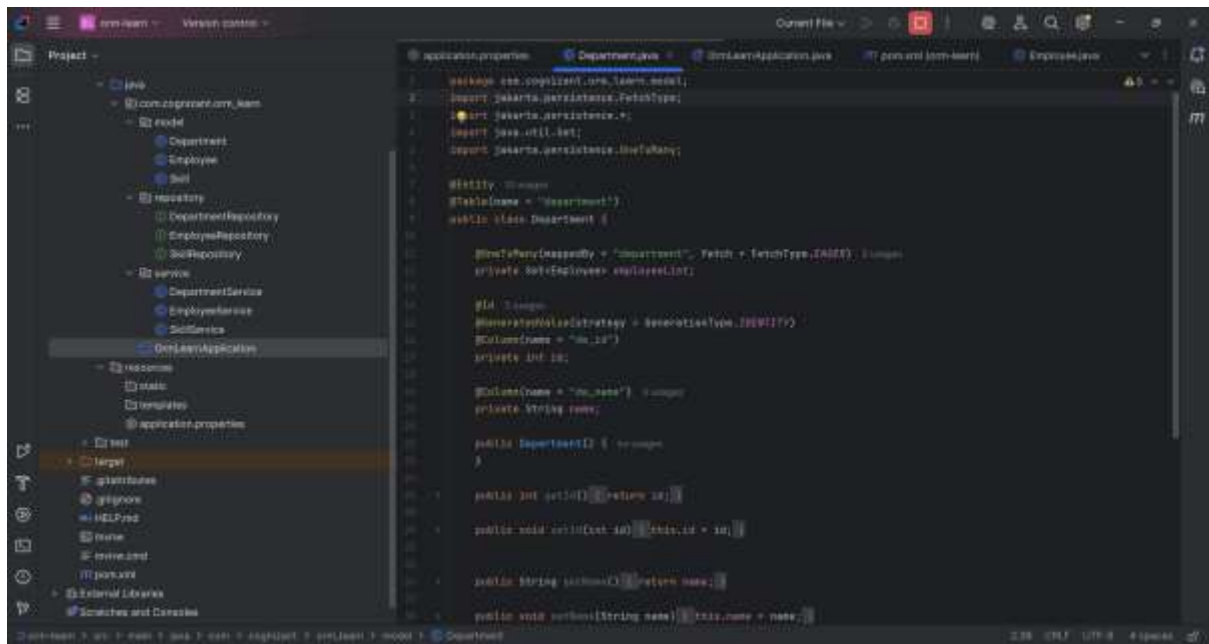
```
2024-06-29T19:15:47.629+05:30 TRACE 27488 --- [orm-learn] [main] org.hibernate.orm.jdbc.bind : binding parameter (1:INTEGER) :- [1]
2024-06-29T19:15:47.636+05:30 INFO 27488 --- [orm-learn] [main] o.o.orm.learn.service.EmployeeService : Start
2024-06-29T19:15:47.657+05:30 DEBUG 27488 --- [orm-learn] [main] org.hibernate.SQL :
select
  e1.e.emp_id,
  e1.e.emp_date_of_birth,
  d1.d.de_id,
  d1.d.de_name,
  e1.e.emp_name,
  e1.e.emp_permanent,
  e1.e.emp_salary
from
  employee e1,e
left join
  department d1,d
  on d1.d.de_id=e1.e.de_id
where
  e1.e.emp_id=1
Hibernate:
select
  e1.e.emp_id,
  e1.e.emp_date_of_birth,
  d1.d.de_id,
  d1.d.de_name,
  e1.e.emp_name,
  e1.e.emp_permanent,
```

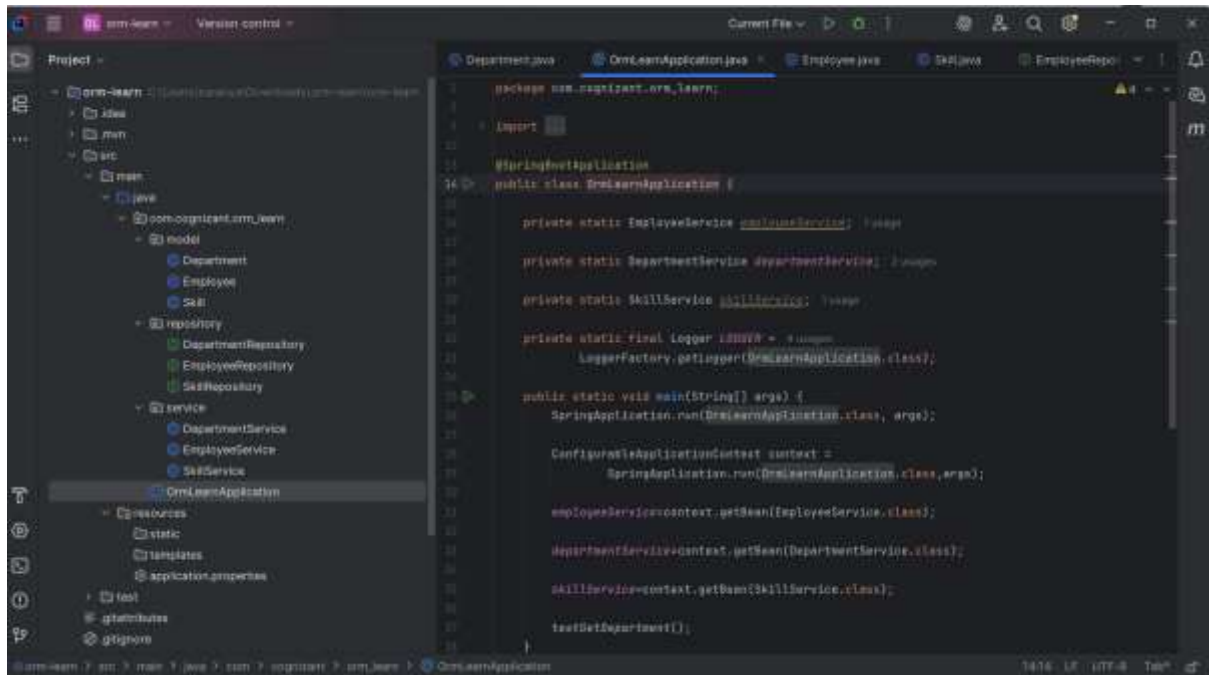
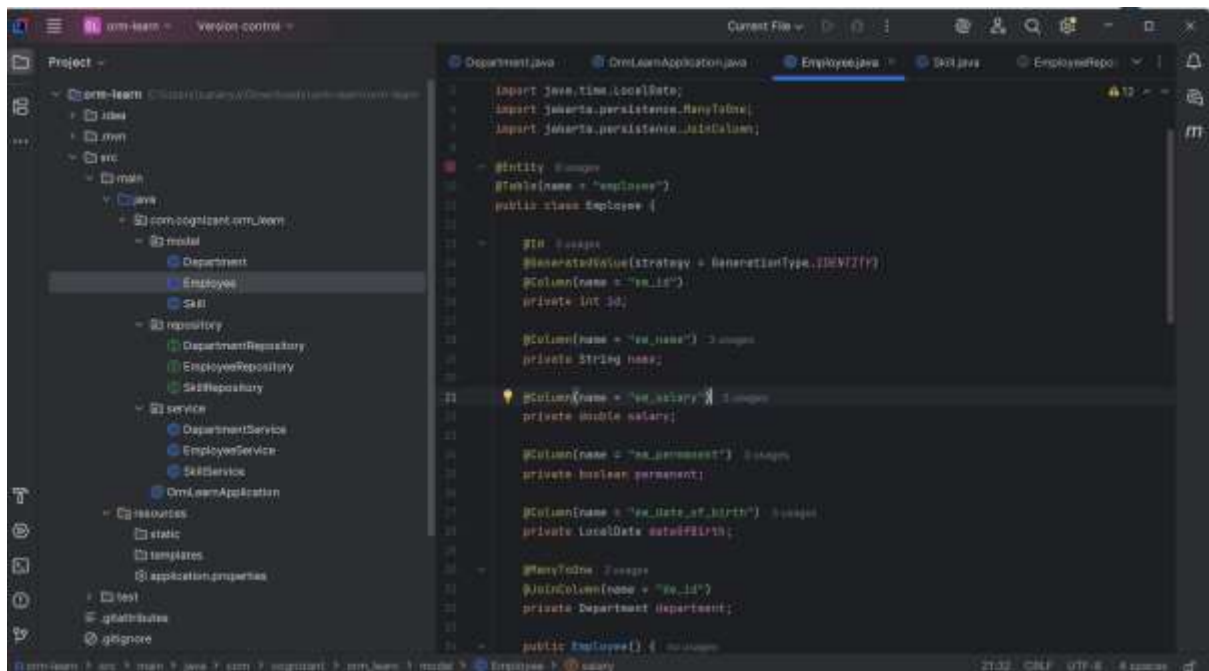
Bottom Screenshot: Java Code

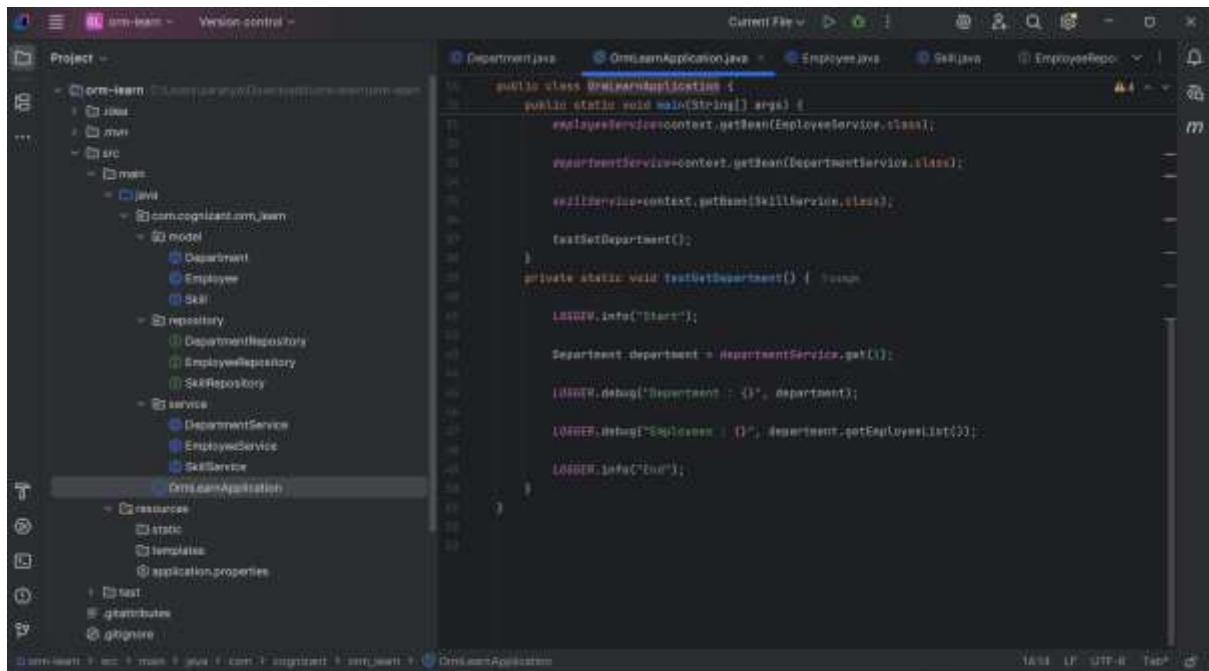
```
org.hibernate.orm.jdbc.bind : binding parameter (1:DATE) :- [1999-03-15]
org.hibernate.orm.jdbc.bind : binding parameter (2:INTEGER) :- [2]
org.hibernate.orm.jdbc.bind : binding parameter (3:VARCHAR) :- [Alice Johnson]
org.hibernate.orm.jdbc.bind : binding parameter (4:BOOLEAN) :- [true]
org.hibernate.orm.jdbc.bind : binding parameter (5:DOUBLE) :- [85000.0]
org.hibernate.orm.jdbc.bind : binding parameter (6:INTEGER) :- [1]
o.o.orm.learn.OrmLearnApplication : Updated Employee : Employee{10-1, name='Alice Johnson', salary=85000.0, permanent=true, dateOfBirth=1999-03-15}
o.o.orm.learn.OrmLearnApplication : End
```

Hands on 5

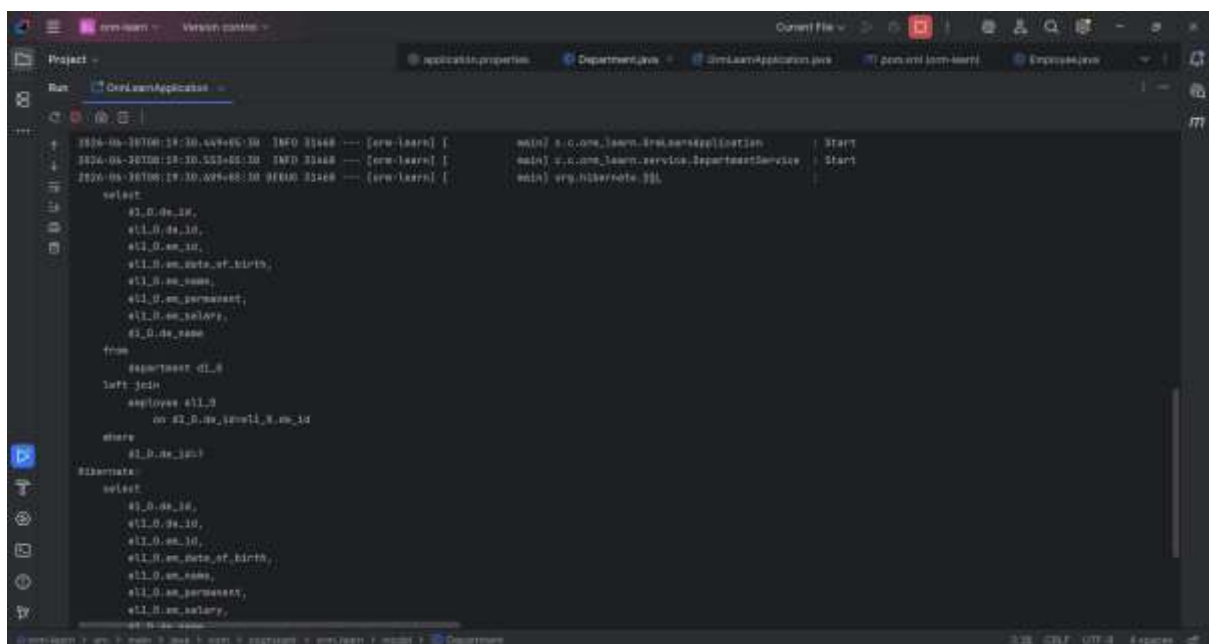
Implement one to many relationship between Employee and Department

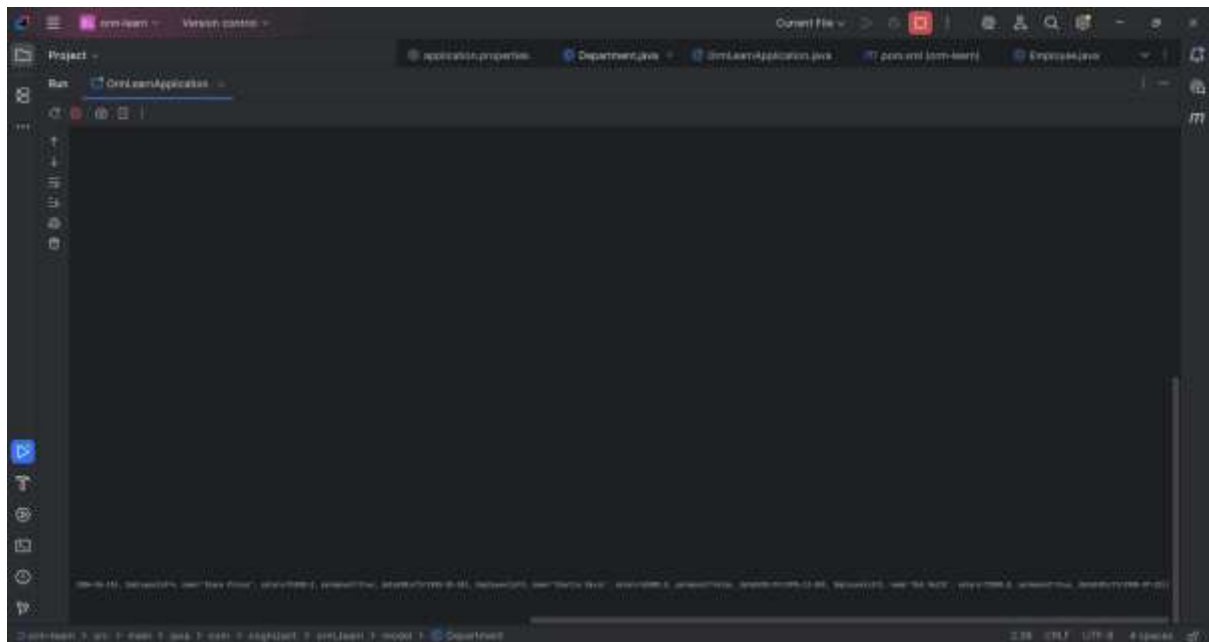






OUTPUT:





Hands on 6

Implement many to many relationship between Employee and Skill

